

0. Lab0 环境搭建与准备工作

在 Microsoft Store 中下载 Ubuntu 22.04.3 LTS, 对软件源进行更新: `sudo apt-get update`
&& `sudo apt-get upgrade`

安装环境: `sudo apt-get install git build-essential gdb-multiarch qemu-system-misc gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu`

测试安装是否成功:

`qemu-system-riscv64 --version`

`riscv64-linux-gnu-gcc --version`

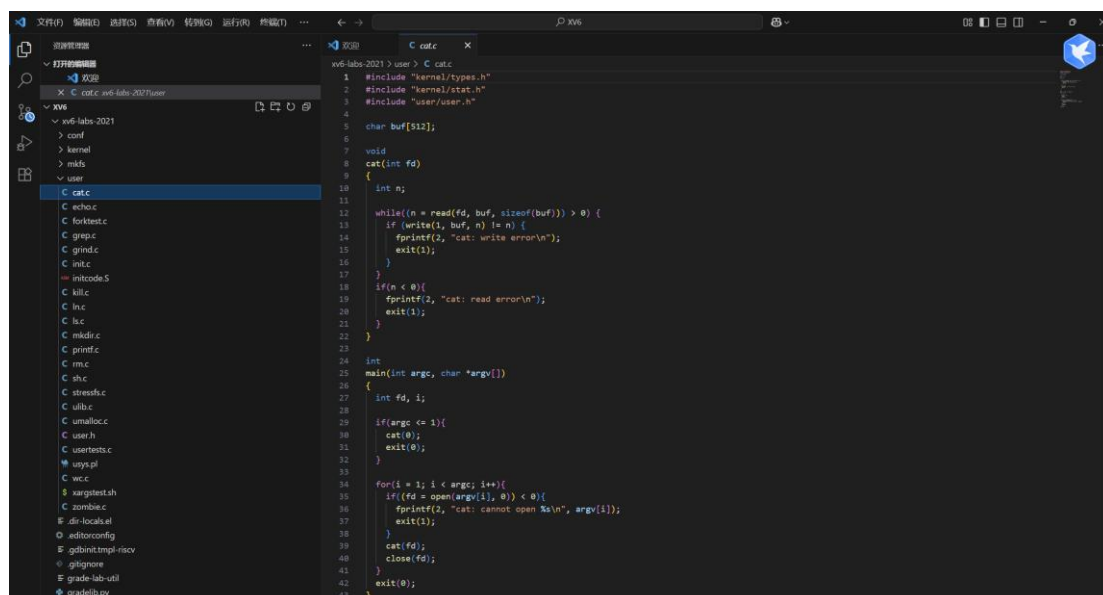
```
canyon@Rikka:~$ qemu-system-riscv64 --version
QEMU emulator version 4.2.1 (Debian 1:4.2-3ubuntu6.30)
Copyright (c) 2003-2019 Fabrice Bellard and the QEMU Project developers
canyon@Rikka:~$ riscv64-linux-gnu-gcc --version
riscv64-linux-gnu-gcc (Ubuntu 9.4.0-1ubuntu1~20.04) 9.4.0
Copyright (C) 2019 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

与 vscode 建立远程连接, 现在 Ubuntu 安装 ssh 服务: `sudo apt-get install openssh-server`

与 vscode 建立远程连接测试 ssh 是否安装成功: `ssh -v`

```
canyon@Rikka:~$ ssh -v
usage: ssh [-46AaCfGgKkMNnqsTtVvXxYy] [-B bind interface]
          [-b bind_address] [-c cipher_spec] [-D [bind_address:]port]
          [-E log_file] [-e escape_char] [-F configfile] [-I pkcs11]
          [-i identity_file] [-J [user@]host[:port]] [-L address]
          [-l login_name] [-m mac_spec] [-O ctl_cmd] [-o option] [-p port]
          [-Q query_option] [-R address] [-S ctl_path] [-W host:port]
          [-w local_tun[:remote_tun]] destination [command]
```

本次实验我选择的是 2021 版的 xv6 labs, 通过下面的命令来复制仓库: `git clone git://g.csail.mit.edu/xv6-labs-2021`



1. Lab1 Xv6 and Unix utilities

1.1. 实验目的

本实验旨在通过实际操作 MIT 6.S081 实验 1, 熟悉 Xv6 操作系统及其基本的 Unix 工具的实现。通过构建和测试一系列基本的 Unix 工具, 理解操作系统的基础概念及进程间通信的机制。

1.2. 实验内容

本实验由五个子实验组成, 每个子实验分别实现了以下功能:

启动并测试 Xv6 操作系统。

实现 sleep 命令: 暂停系统运行指定的时间。

实现 pingpong 命令: 父子进程间通过管道进行通信, 模拟“乒乓”传递。

实现 primes 命令: 通过筛法找出一定范围内的质数, 并使用多进程实现并行计算。

实现 find 和 xargs 命令: 查找特定目录下的文件, 并根据输入生成命令行参数。

1.3. 实验步骤

Boot xv6

1、获取实验用的 Xv6 源码并切换到 util 分支:

2、在 Xv6 中测试基本命令:

使用 ls 列出文件。

使用 Ctrl+p 查看运行进程。

使用 Ctrl+a x 退出 QEMU。

```
canyon@Rikka:~/xv6-labs-2021$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

sleep

1、编写 sleep.c 文件, 添加功能:

接受一个参数, 该参数为睡眠时间 (以时钟滴答为单位)。处理错误输入, 如参数数量不正确或参数不是有效数字。

```
C sleep.c U X M Makefile M
xv6-labs-2021 > user > C sleep.c
1 #include "kernel/types.h"
2 #include "kernel/stat.h"
3 #include "user/user.h"
4
5 int
6 main(int argc, char *argv[])
7 {
8     // 检查参数数量是否正确
9     // sleep 程序需要一个参数: 睡眠的 tick 数
10    if(argc != 2){
11        // 如果参数数量不正确, 输出错误信息并退出
12        fprintf(2, "用法: sleep <ticks>\n");
13        exit(1);
14    }
15
16    // 将命令行参数转换为整数
17    // atoi 函数将字符串转换为整数
18    int ticks = atoi(argv[1]);
19
20    // 检查参数是否为有效数字 (非负数)
21    if(ticks < 0){
22        fprintf(2, "错误: tick 数必须为非负数\n");
23        exit(1);
24    }
25
26    // 调用 sleep 系统调用
27    // sleep 系统调用接受一个整数参数, 表示要睡眠的 tick 数
28    // 一个 tick 是 xv6 内核定义的时间单位, 即定时器芯片两次中断之间的时间
29    sleep(ticks);
30
31    // 睡眠完成后正常退出
32    exit(0);
33 }
34 | Ctrl+L to chat, Ctrl+K to generate
```

2、修改 Makefile 文件:

```
C sleep.c U M Makefile M X
xv6-labs-2021 > M Makefile
179 UPROGS=\
181     $U/_echo\
182     $U/_forktest\
183     $U/_grep\
184     $U/_init\
185     $U/_kill\
186     $U/_ln\
187     $U/_ls\
188     $U/_mkdir\
189     $U/_rm\
190     $U/_sh\
191     $U/_sleep\
192     $U/_stressfs\
193     $U/_usertests\
194     $U/_grind\
195     $U/_wc\
196     $U/_zombie\
```

将 sleep 程序添加到 UPROGS 部分。

3、测试 sleep 命令:

在 Xv6 中运行并测试 sleep 命令的功能。

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 10
```

pingpong

1、编写 pingpong.c 文件, 添加功能:

创建两个管道用于父子进程间的通信。子进程从父进程接收数据后打印 ping, 然后将数据传回给父进程。父进程接收到数据后打印 pong。

```

xv6-labs-2021 > user > C pingpong.c
1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  int
6  main(int argc, char *argv[])
7  {
8      // 检查参数数量, pingpong 程序不需要参数
9      if(argc != 1){
10         fprintf(2, "用法: pingpong\n");
11         exit(1);
12     }
13
14     // 创建两个管道
15     // pipe1: 父进程 -> 子进程
16     // pipe2: 子进程 -> 父进程
17     int pipe1[2]; // pipe1[0] 是读端, pipe1[1] 是写端
18     int pipe2[2]; // pipe2[0] 是读端, pipe2[1] 是写端
19
20     if(pipe(pipe1) < 0 || pipe(pipe2) < 0){
21         fprintf(2, "pingpong: pipe 创建失败\n");
22         exit(1);
23     }
24
25     // 创建子进程
26     int pid = fork();
27
28     if(pid < 0){
29         // fork 失败
30         fprintf(2, "pingpong: fork 失败\n");
31         exit(1);
32     } else if(pid == 0){
33         // 子进程/child

```

2、修改 Makefile 文件:

将 pingpong 程序添加到 UPROGS 部分。

3、测试 pingpong 命令:

在 Xv6 中运行并测试 pingpong 命令, 确保父子进程的通信正常。

```

canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-util pingpong
make: 'kernel/kernel' is up to date.
== Test pingpong == pingpong: OK (0.9s)

canyon@Rikka:~/xv6-labs-2021$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -drive file=fs.img,if=none,format
=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ pingpong
4: received ping
3: received pong

```

primes

1、编写 primes.c 文件, 添加功能:

使用管道和多进程实现“筛法”来找出 2 到 35 之间的所有质数。每个进程通过管道接收数值, 并判断该数值是否能被已找到的质数整除, 不能整除的数值则传递给下一个进程。

```
C sleep.c U  C primes.c U X
xv6-labs-2021 > user > C primes.c
1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  // 素数筛法的并发实现
6  // 每个进程接收一个数字作为素数，然后过滤掉所有能被该素数整除的数字
7  // 每个进程创建一个子进程来处理下一个素数
8
9  void
10 prime_sieve(int read_pipe)
11 {
12     int prime;
13     int n;
14
15     // 从父进程读取第一个数字作为素数
16     if(read(read_pipe, &prime, sizeof(prime)) != sizeof(prime)){
17         // 如果没有数字可读，说明已经结束
18         close(read_pipe);
19         exit(0);
20     }
21
22     // 打印找到的素数
23     printf("prime %d\n", prime);
24
25     // 创建管道用于与子进程通信
26     int pipe_fd[2];
27     if(pipe(pipe_fd) < 0){
28         fprintf(2, "primes: pipe 创建失败\n");
29         exit(1);
30     }
31
32     // 创建子进程
33     int pid = fork();
```

2、修改 Makefile 文件：

将 primes 程序添加到 UPROGS 部分。

3、测试 primes 命令：

在 Xv6 中运行并测试 primes 命令，确保能正确输出所有质数。

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ primes
prime 2
prime 3
prime 5
prime 7
prime 11
prime 13
prime 17
prime 19
prime 23
prime 29
prime 31
```

find

1、编写 find.c 文件，添加功能：

实现递归遍历目录，查找与指定文件名匹配的文件。使用 struct dirent 和 struct stat 进行文件及目录的处理。

```
C sleep.c U C primes.c U C find.c U X
xv6-labs-2021 > user > C find.c
1 #include "kernel/types.h"
2 #include "kernel/stat.h"
3 #include "user/user.h"
4 #include "kernel/fs.h"
5
6 // 递归查找函数
7 // path: 当前搜索路径
8 // target: 要查找的文件名
9 void
10 find(char *path, char *target)
11 {
12     char buf[512], *p;
13     int fd;
14     struct dirent de;
15     struct stat st;
16
17     // 打开当前目录
18     if((fd = open(path, 0)) < 0){
19         fprintf(2, "find: cannot open %s\n", path);
20         return;
21     }
22
23     // 获取当前目录的状态信息
24     if(fstat(fd, &st) < 0){
25         fprintf(2, "find: cannot stat %s\n", path);
26         close(fd);
27         return;
28     }
29
30     // 检查当前路径是否为目录
```

- 2、修改 Makefile 文件：
将 find 程序添加到 UPROGS 部分。
- 3、测试 find 命令：
在 Xv6 中运行并测试 find 命令，验证其功能。

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ echo > b
$ mkdir a
$ echo > a/b
$ find . b
./b
./a/b
```

xargs

- 1、编写 xargs.c 文件，添加功能：
接收标准输入作为命令行参数，并生成命令执行。处理多个参数及空格、换行等特殊字符。

```

C xargs.c U X
xv6-labs-2021 > user > C xargs.c
1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  #define MAXARG 32 // 最大参数数量
6  #define MAXLINE 512 // 最大行长度
7
8  // 运行命令函数
9  // argv: 命令参数数组
10 // Line: 从标准输入读取的行 (作为额外参数)
11 void
12 run_command(char *argv[], char *line)
13 {
14     // 去除行末尾的换行符
15     int len = strlen(line);
16     if(len > 0 && line[len-1] == '\n'){
17         line[len-1] = '\0';
18     }
19
20     // 计算原始参数数量
21     int argc = 0;
22     while(argv[argc] != 0){
23         argc++;
24     }
25
26     // 将行内容作为额外参数添加到命令中
27     if(len > 1){ // 如果行不为空 (去除换行符后)
28         argv[argc] = line;
29         argv[argc + 1] = 0; // 添加字符串结束符
30     }

```

2、修改 Makefile 文件：

将 xargs 程序添加到 UPROGS 部分。

3、测试 xargs 命令：

在 Xv6 中运行并测试 xargs 命令，验证其功能。

```

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ sh < xargstest.sh
$ $ $ $ $ $ hello
hello
hello

```

4、使用 make grade 进行最终的测试：

```

canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-util sleep
make: 'kernel/kernel' is up to date.
== Test sleep, no arguments == sleep, no arguments: OK (0.7s)
== Test sleep, returns == sleep, returns: OK (0.9s)
== Test sleep, makes syscall == sleep, makes syscall: OK (0.9s)
canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-util primes
make: 'kernel/kernel' is up to date.
== Test primes == primes: OK (0.9s)
canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-util find
make: 'kernel/kernel' is up to date.
== Test find, in current directory == find, in current directory: OK (1.5s)
== Test find, recursive == find, recursive: OK (0.9s)
canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-util xargs
make: 'kernel/kernel' is up to date.
== Test xargs == xargs: OK (1.3s)
(Old xv6.out.xargs failure log removed)

```

1.4. 实验心得

通过本次实验，我深入理解了操作系统的基本原理，尤其是在进程管理和进程间通信方面。通过实现和测试这些基本的 Unix 工具，我不仅熟悉了 C 语言的系统编程，还学会了

如何使用进程和管道来实现进程间的数据传递。同时，实验也让我认识到，尽管这些工具看似简单，但其背后的实现涉及到系统资源的管理和优化，这对于理解和设计一个操作系统至关重要。

此外，在编写和调试代码的过程中，我深刻体会到细致的代码审查和测试的重要性，尤其是在处理复杂的递归逻辑和进程通信时，稍有疏忽就可能导致程序的意外行为。这也提醒我在实际开发中要保持严谨和耐心，以确保代码的正确性和稳定性。

2. Lab2 System Calls

2.1. 实验目的

本实验的目的是通过修改 xv6 操作系统内核，实现和测试特定的系统调用。主要任务包括实现系统调用追踪 (trace) 功能和实现系统信息查询 (sysinfo) 功能。这些任务要求我们深入理解 xv6 的内核结构，并通过编写和测试系统调用，进一步掌握内核开发的技能。

2.2. 实验内容

系统调用追踪 (trace) 功能：

通过实现 trace 系统调用，可以追踪指定的系统调用并在系统调用返回时打印相关信息，包括进程 ID、系统调用名称以及返回值。实验要求我们对 xv6 内核进行必要的修改，以支持 trace 功能。

系统信息查询 (sysinfo) 功能：

实现 sysinfo 系统调用，使得用户程序能够查询系统的空闲内存量和当前正在运行的进程数量。该任务要求我们计算内存和进程的信息，并将其通过系统调用返回给用户态程序。

2.3. 实验步骤

System call tracing

1、在进程结构体中添加跟踪标志：

```
99 // these are private to the process, so p->lock need not be held.
100 uint64 kstack;           // Virtual address of kernel stack
101 uint64 sz;               // Size of process memory (bytes)
102 pagetable_t pagetable;   // User page table
103 struct trapframe *trapframe; // data page for trampoline.S
104 struct context context;   // swtch() here to run process
105 struct file *ofile[NOFILE]; // Open files
106 struct inode *cwd;        // Current directory
107 char name[16];           // Process name (debugging)
108 int trace_mask;          // 跟踪掩码，用于控制哪些系统调用需要被跟踪
109 };
```

2、在 syscall.h 中添加 trace 系统调用的定义

```
#define SYS_trace 22
```


3、在 syscall.c 中添加 trace 系统调用的实现

```
extern uint64 sys_trace(void);  
[SYS_trace] sys_trace,
```

4、修改 syscall 函数来添加跟踪功能：

```
// 系统调用名称数组，用于跟踪输出  
static char *syscall_names[] = {  
    [SYS_fork]    "fork",  
    [SYS_exit]    "exit",  
    [SYS_wait]    "wait",  
    [SYS_pipe]    "pipe",  
    [SYS_read]    "read",  
    [SYS_kill]    "kill",  
    [SYS_exec]    "exec",  
    [SYS_fstat]   "fstat",  
    [SYS_chdir]   "chdir",  
    [SYS_dup]     "dup",  
    [SYS_getpid]  "getpid",  
    [SYS_sbrk]    "sbrk",  
    [SYS_sleep]   "sleep",  
    [SYS_uptime]  "uptime",  
    [SYS_open]    "open",  
    [SYS_write]   "write",  
    [SYS_mknod]   "mknod",  
    [SYS_unlink]  "unlink",  
    [SYS_link]    "link",  
    [SYS_mkdir]   "mkdir",  
    [SYS_close]   "close",  
    [SYS_trace]   "trace",  
};
```

5、在 sysproc.c 中添加 trace 系统调用的实现：

```
// trace 系统调用实现  
// 设置当前进程的跟踪掩码，控制哪些系统调用需要被跟踪  
uint64  
sys_trace(void)  
{  
    int mask;  
  
    // 获取跟踪掩码参数  
    if(argint(0, &mask) < 0)  
        return -1;  
  
    // 设置当前进程的跟踪掩码  
    struct proc *p = myproc();  
    p->trace_mask = mask;  
  
    return 0;  
}
```

7、更新 Makefile：

在 Makefile 中添加 \$U/_trace 到 UPROGS 中，以便测试 trace 功能。

8、测试：

xv6 中执行相关测试：

```

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 968
3: syscall read -> 235
3: syscall read -> 0
$
$ grep hello README
$
$ trace 2 usertests forkforkfork
usertests starting
7: syscall fork -> 8
test forkforkfork: 7: syscall fork -> 9
9: syscall fork -> 10
10: syscall fork -> 11
10: syscall fork -> 12
11: syscall fork -> 13
10: syscall fork -> 14
12: syscall fork -> 15
13: syscall fork -> 16
11: syscall fork -> 17
10: syscall fork -> 18
13: syscall fork -> 19

```

./grade-lab-syscall trace 单项测试:

```

riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_SYSCALL -DLAB_SYSCALL -MD -march=rv64g
medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -march=rv64g
-nostdinc -I. -Ikernel -c user/initcode.S -o user/initcode.o
riscv64-linux-gnu-ld -z max-page-size=4096 -N -e start -Ttext 0 -o user/initcode.out user/initcode.o
riscv64-linux-gnu-objcopy -S -O binary user/initcode.out user/initcode
riscv64-linux-gnu-objdump -S user/initcode.o > user/initcode.asm
riscv64-linux-gnu-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/kalloc.o
kernel/string.o kernel/main.o kernel/vm.o kernel/proc.o kernel/swtch.o kernel/trampoline.o kernel/trap.o kern
el/syscall.o kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o kernel/pi
pe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/plic.o kernel/virtio_disk.o kernel/start.o kerne
l/console.o kernel/printf.o kernel/uart.o kernel/spinlock.o
riscv64-linux-gnu-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-linux-gnu-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kernel/kernel.sym
== Test trace 32 grep == trace 32 grep: OK (2.5s)
== Test trace all grep == trace all grep: OK (0.8s)
== Test trace nothing == trace nothing: OK (0.9s)
== Test trace children == trace children: OK (7.7s)
canyon@Rikka:~/xv6-labs-2021$

```

Sysinfo

- 1、在系统调用头文件中添加 sysinfo 定义

```
#define SYS_sysinfo 23
```

2、在 syscall.c 中添加 sysinfo 声明和实现

```
extern uint64 sys_sysinfo(void);
```

```
SYS_sysinfo] sys_sysinfo,
```

3、在 sysproc.c 中实现 sysinfo 系统调用

```
Makefile M x C trace.c M C syscall.c M C sysproc.c M x
xv6-labs-2021 > kernel > C sysproc.c
102 sys_trace(void)
103 {
104     int mask;
105
106     // 获取跟踪掩码参数
107     if(argint(0, &mask) < 0)
108         return -1;
109
110     // 设置当前进程的跟踪掩码
111     struct proc *p = myproc();
112     p->trace_mask = mask;
113
114     return 0;
115 }
116
117 // sysinfo 系统调用实现
118 // 收集系统信息：空闲内存数量和活跃进程数量
119 uint64
120 sys_sysinfo(void)
121 {
122     uint64 addr;
123     struct sysinfo info;
124
125     // 获取用户空间 sysinfo 结构体的地址
126     if(argaddr(0, &addr) < 0)
127         return -1;
128
129     // 初始化 sysinfo 结构体
130     info.freemem = kfreemem(); // 获取空闲内存数量
131     info.nproc = nproc();     // 获取活跃进程数量
```

4、实现 kfreemem() 和 nproc() 函数

```
// 计算空闲内存数量（字节）
uint64
kfreemem(void)
{
    struct run *r;
    uint64 free_mem = 0;

    acquire(&kmem.lock);
    r = kmem.freelist;
    while(r){
        free_mem += PGSIZE;
        r = r->next;
    }
    release(&kmem.lock);

    return free_mem;
}
```

```
// 计算活跃进程数量（状态不是 UNUSED 的进程）
uint64
nproc(void)
{
    struct proc *p;
    uint64 count = 0;

    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->state != UNUSED){
            count++;
        }
        release(&p->lock);
    }

    return count;
}
```

5、更新 Makefile:

在 Makefile 中添加 `$U/_sysinfotest` 到 `UPROGS` 中，以便测试 `sysinfo` 功能。

6、测试:

在 Xv6 中测试:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sysinfotest
sysinfotest: start
sysinfotest: OK
$ QEMU: Terminated
```

通过执行 `./grade-lab-syscall sysinfotest` 来测试 `sysinfo` 功能。

```
canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-syscall sysinfotest
make: 'kernel/kernel' is up to date.
== Test sysinfotest == sysinfotest: OK (2.8s)
```

2.4. 实验心得

通过本次实验，我对 xv6 操作系统的内核结构和系统调用机制有了更深入的理解。实验中的每一个步骤都要求我们仔细阅读和修改内核代码，这不仅增强了对 C 语言和内核编程的掌握，还培养了我们调试和解决问题的能力。

特别是在 `trace` 功能的实现过程中，我体会到了内核中位操作和掩码使用的重要性。同时，`sysinfo` 功能的实现让我理解了如何在内核态与用户态之间传递数据，以及如何合理计算和维护系统资源信息。

整个实验过程虽然充满挑战，但每一步的成功都带来了极大的成就感。这不仅加深了我对操作系统内核的理解，也为以后从事内核相关的开发工作打下了坚实的基础。

3. Lab3 Page Tables

3.1. 实验目的

探索页表并对其进行修改以加快某些系统调用，并检测哪些页已被访问。

3.2. 实验内容

本实验包含以下几个部分：

加速系统调用：通过将数据映射到用户空间的只读区域，减少内核与用户空间之间的切换，从而加速某些系统调用。

打印页表：实现一个函数以打印特定进程的页表内容，帮助学生理解页表的结构并为未来的调试提供支持。

检测页面访问：通过解析 RISC-V 的页表，检测哪些页面被访问过，从而为垃圾回收等内存管理任务提供支持。

3.3. 实验步骤

Speed up system calls

1、阅读相关资料

在开始编写代码之前，首先阅读 xv6 的第 3 章以及以下文件：

kern/memlayout.h：内存布局定义。

kern/vm.c：虚拟内存相关代码。

kernel/kalloc.c：物理内存分配与释放代码。

2、修改 proc_pagetable 函数

在 kernel/proc.c 中的 proc_pagetable 函数中，将一页只读页面映射到 USYSCALL，并在页面的起始处存储 struct usyscall，初始化时将当前进程的 PID 存入其中。

```
xv6-labs-2021 > kernel > C proc.h
86  struct proc {
102      pagetable_t pagetable;      // User page table
103      struct trapframe *trapframe; // data page for trampoline.S
104      struct usyscall *usyscall;  // lab3 data page for usyscall
```

```
xv6-labs-2021 > kernel > C proc.c
185  {
210
211      // lab3: map the usyscall just below TRAMPOLINE
212      if(mappages(pagetable, USYSCALL, PGSIZE,
213                  (uint64)(p->usyscall), PTE_R | PTE_U) < 0){
214          uvmunmap(pagetable, USYSCALL, 1, 0);
215          uvmunmap(pagetable, TRAMPOLINE, 1, 0);
216          uvmfree(pagetable, 0);
217          return 0;
218      }
```

3、分配和释放页面

在 allocproc()中分配该页面，并在 freeproc()中释放该页面，确保资源不会泄漏。

```
xv6-labs-2021 > kernel > C proc.c
119  found:
120  }
129
130      // lab3 Allocate a usyscall page.
131      if((p->usyscall = (struct usyscall *)kalloc()) == 0){
132          freeproc(p);
133          release(&p->lock);
134          return 0;
135      }
136      p->usyscall->pid = p->pid;
```

```
xv6-labs-2021 > kernel > C proc.c
159  freeproc(struct proc *p)
160  {
161      if(p->trapframe)
162          kfree((void*)p->trapframe);
163      p->trapframe = 0;
164      //lab3 free usyscall page
165      if(p->usyscall)
166          kfree((void*)p->usyscall);
167      p->usyscall = 0;
```

allocproc 会调用 proc_pagetable 函数，用于创建进程的页表，并将几个特殊页面（trampoline、trapframe、加速页面）映射到页表中。需要修改 proc_freepagetable()函数的逻辑。

```
xv6-labs-2021 > kernel > C proc.c
222
223  // Free a process's page table, and free the
224  // physical memory it refers to.
225  void
226  proc_freepagetable(pagetable_t pagetable, uint64 sz)
227  {
228      // lab3:unmap usyscall page
229      uvmunmap(pagetable, USYSCALL, 1, 0);
230      uvmunmap(pagetable, TRAMPOLINE, 1, 0);
231      uvmunmap(pagetable, TRAPFRAME, 1, 0);
232      uvmfree(pagetable, sz);
233  }
```

4、运行测试

运行 pgtbltest 来验证 ugetpid()是否能够正确获取 PID，确保加速系统调用部分的功能实现。

```
canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-pgtbl ugetpid
make: 'kernel/kernel' is up to date.
== Test pgtbltest == (1.3s)
== Test pgtbltest: ugetpid ==
pgtbltest: ugetpid: OK
```

Print a page table

1、定义 vmprint 函数

在 kernel/vm.c 中实现 vmprint()函数，用于打印给定的页表。函数需接收一个 pagetable_t 类型的参数，并按照指定格式打印页表内容。（记得添加函数原型）

```
xv6-labs-2021 > kernel > C vm.c
435
436 // lab3 递归打印页表信息
437 void vmprint(pagetable_t pagetable) {
438     printf("page table %p\n", pagetable); // 打印顶级页表的基地址
439
440     const int PAGE_SIZE = 512; // 定义每级页表的条目数量
441
442     // 遍历顶级页目录
443     for (int i = 0; i < PAGE_SIZE; ++i) {
444         pte_t top_pte = pagetable[i];
445
446         if (top_pte & PTE_V) { // 如果页表项有效
447             printf("..%d: pte %p pa %p\n", i, top_pte, PTE2PA(top_pte));
448
449             pagetable_t mid_table = (pagetable_t) PTE2PA(top_pte); // 获取下一级页表的地址
450
451             // 递归打印子目录
452             vmprint(mid_table);
453         }
454     }
455 }

xv6-labs-2021 > kernel > C defs.h
172 int copyinstr(pagetable_t, char *, uint64, uint64);
173 void vmprint(pagetable_t pagetable); // lab3 函数原型
```

2、修改 exec.c

在 exec.c 中的 exec() 函数的最后，插入 if(p->pid==1) vmprint(p->pagetable)，打印第一个进程的页表。

```
xv6-labs-2021 > kernel > C exec.c
14 {
118
119 // lab3 打印页表
120 if(p->pid==1)
121     vmprint(p->pagetable);
122
123 return argc; // this ends up in a0, the first argument to main(argc, argv)
```

3、运行测试

编译并运行 xv6，检查是否能够正确打印出页表内容，验证页表打印功能是否正常。

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
page table 0x0000000087f6e000
..0: pte 0x0000000021fda801 pa 0x0000000087f6a000
.. ..0: pte 0x0000000021fda401 pa 0x0000000087f69000
.. .. ..0: pte 0x0000000021fdac1f pa 0x0000000087f6b000
.. .. ..1: pte 0x0000000021fda00f pa 0x0000000087f68000
.. .. ..2: pte 0x0000000021fd9c1f pa 0x0000000087f67000
..255: pte 0x0000000021fdb401 pa 0x0000000087f6d000
.. ..511: pte 0x0000000021fdb001 pa 0x0000000087f6c000
.. .. ..509: pte 0x0000000021fdd813 pa 0x0000000087f76000
.. .. ..510: pte 0x0000000021fddc07 pa 0x0000000087f77000
.. .. ..511: pte 0x0000000020001c0b pa 0x0000000080007000
init: starting sh
```

用命令 ./grade-lab-pgtbl pte 进行测试

```
canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-pgtbl pte
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_PGTBL -DLAB_PGTBL -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/vm.o kernel/vm.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/kalloc.o kernel/string.o kernel/main.o kernel/vm.o kernel/proc.o kernel/swtch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/plic.o kernel/virtio_disk.o kernel/start.o kernel/console.o kernel/printf.o kernel/uart.o kernel/spinlock.o
riscv64-linux-gnu-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-linux-gnu-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kernel/kernel.sym
== Test pte printout == pte printout: OK (1.3s)
(Old xv6.out.pteprint failure log removed)
```

4、分析页表输出

根据实验手册中的示例输出分析页表内容，理解每一层页表项所指向的物理页面。

Detecting which pages have been accessed

1、实现 pgaccess 系统调用

在 kernel/sysproc.c 中实现 sys_pgaccess() 函数，该函数接收起始虚拟地址、页面数量以及结果缓冲区地址，将页面访问情况存储在结果缓冲区中。

```
xv6-labs-2021 > kernel > C sysproc.c
79  #ifdef LAB_PGTBL
80  int
81  sys_pgaccess(void)
82  {
83      // lab pgtbl: your code here.
84      // lab3
85      uint64 addr;
86      int len;
87      int bitmask;
88      if(argaddr(0, &addr) < 0)
89          return -1;
90      if(argint(1, &len) < 0)
91          return -1;
92      if(argint(2, &bitmask) < 0)
93          return -1;
94
95      if(len > 32 || len < 0){
96          return -1;
97      }
98
99      int res = 0;
100     struct proc *p = myproc();
101     //算res
102     for(int i = 0; i < len; i++){
103         int va = addr + i * PGSIZE;
104         int abit = vm_pgaccess(p->pagetable, va);
105         res = res | abit << i;
106     }
107
108     if(copyout(p->pagetable, bitmask, (char*)&res, sizeof(res)) < 0){
109         return -1;
110     }
111 }
```

修改宏定义，加入一个 PTE_A 位。


```
xv6-labs-2021 > kernel > C riscv.h
343 #define PTE_W (1L << 2)
344 #define PTE_X (1L << 3)
345 #define PTE_U (1L << 4) // 1 -> user can access
346 | #define PTE_A (1L << 6) // lab3: define PTE_A
```

继续修改 sys_pgaccess():

```
xv6-labs-2021 > kernel > C vm.c
467 // lab3: pgaccess
468 int vm_pgaccess(pagetable_t pagetable, uint64 va){
469     pte_t *pte;
470
471     if(va >= MAXVA)
472         return 0;
473
474     pte = walk(pagetable, va, 0);
475     if(pte == 0)
476         return 0;
477     if((*pte & PTE_A) != 0){
478         *pte = *pte & (~PTE_A); // 第六位归零 (PTE_A)
479         return 1;
480     }
481     return 0;
482 }
```

将函数原型加入到 defs.h。

```
xv6-labs-2021 > kernel > C defs.h
173 void vmprint(pagetable_t pagetable); // lab3
174 | int vm_pgaccess(pagetable_t pagetable, uint64 va); // lab3
```

2、解析参数和填充位掩码

使用 `argaddr()` 和 `argint()` 解析系统调用参数，并在内核中构建临时缓冲区，用于存储位掩码。最后，利用 `copyout()` 将位掩码复制到用户空间。

3、检查和清除访问位

利用 `walk()` 函数找到相关 PTE，检查其访问位是否被置位。检测后，清除访问位以便下次调用时能检测到新访问的页面。

4、运行测试

通过 `pgtbltest` 测试 `pgaccess()` 系统调用，确保该功能能够正确检测并报告页面的访问情况。

```
hart 1 starting
hart 2 starting
page table 0x0000000087f6e000
..0: pte 0x0000000021fda801 pa 0x0000000087f6a000
.. ..0: pte 0x0000000021fda401 pa 0x0000000087f69000
.. .. ..0: pte 0x0000000021fdac1f pa 0x0000000087f6b000
.. .. ..1: pte 0x0000000021fda00f pa 0x0000000087f68000
.. .. ..2: pte 0x0000000021fd9c1f pa 0x0000000087f67000
..255: pte 0x0000000021fdb401 pa 0x0000000087f6d000
.. ..511: pte 0x0000000021fdb001 pa 0x0000000087f6c000
.. .. ..509: pte 0x0000000021fdd813 pa 0x0000000087f76000
.. .. ..510: pte 0x0000000021fddc07 pa 0x0000000087f77000
.. .. ..511: pte 0x0000000020001c0b pa 0x0000000080007000
init: starting sh
$ pgtbltest
ugetpid_test starting
ugetpid_test: OK
pgaccess_test starting
pgaccess_test: OK
pgtbltest: all tests succeeded
$
```

3.4. 实验心得

本实验完成了有关虚拟内存和页表机制的实验内容。总的来说实验难度都没有很大，在指导书的帮助下可以很快完成，但是重在自己的理解和代码的全局理解。

虚拟内存和页表机制是计算机系统中非常重要的概念。通过使用虚拟内存，操作系统可以屏蔽底层硬件的细节，为每个进程提供独立的地址空间，从而增强了系统的安全性和可靠性。

在实验中遇到了一些问题。比如一个内核级别的错误，错误信息为"pgaccess : walk failed"。这表明在访问页表时发生了错误。最后我重新复制了一下源代码，重启完成。在实验中保持电脑系统的正常也是很重要的。

4. Lab4 Traps

```
xv6-labs-2021 > user > ASM call.asm
13  int g(int x) {
15      2: e422                sd s0,8(sp)
16      4: 0800                addi s0,sp,16
17      return x+3;
18  }
19      6: 250d                addiw a0,a0,3
20      8: 6422                ld s0,8(sp)
21      a: 0141                addi sp,sp,16
22      c: 8082                ret
23
24  000000000000000e <f>:
25
26  int f(int x) {
27      e: 1141                addi sp,sp,-16
28      10: e422                sd s0,8(sp)
29      12: 0800                addi s0,sp,16
30      return g(x);
31  }
32      14: 250d                addiw a0,a0,3
33      16: 6422                ld s0,8(sp)
34      18: 0141                addi sp,sp,16
35      1a: 8082                ret
36
37  000000000000001c <main>:
38
39  void main(void) {
40      1c: 1141                addi sp,sp,-16
41      1e: e406                sd ra,8(sp)
42      20: e022                sd s0,0(sp)
43      22: 0800                addi s0,sp,16
44      printf("%d %d\n", f(8)+1, 13);
45      24: 4635                li a2,13
46      26: 45b1                li a1,12
```

Q1: 函数参数存储在寄存器 a0~a7 中。对于 printf 调用的 13, 它被存储在寄存器 a2 中。

Q2: 在第 40 行, 编译器进行了函数内联优化, 直接计算出 f(8)+1 的值为 12。

Q3: 通过第 43 和 44 行的指令可以看出, jalr 跳转到的地址为 0x30+1536=0x630, 因此函数 printf 的地址是 0x630。

Q4: 根据 jalr 指令的工作原理, 在跳转到 printf 后, ra 寄存器的值为跳转前的 pc+4, 即 0x34+4=0x38。

Q5: 执行以下代码:

```
unsigned int i = 0x00646c72;  
printf("H%x Wo%s", 57616, &i);
```

输出为: HE110 World。

```
xv6 kernel is booting  
  
hart 1 starting  
hart 2 starting  
init: starting sh  
$ call  
12 13  
HE110 World$
```

如果 RISC-V 是大端模式, 需要将 `i` 设置为 `0x726c6400`, 以获得相同的输出结果。而 `57616` 的值不需要改变, 因为它是按二进制数处理的, 而不是按字符读取。

Q6: 对于以下代码:

```
printf("x=%d y=%d", 3);
```

输出 `y=` 后的值将是寄存器 `a2` 的值。这是因为 `printf` 的格式字符串与实际传递的参数数量不匹配, 导致 `printf` 仍然尝试从寄存器中读取预期的参数值。按照 RISC-V 的传参规则, 第二个不定参数会存储在寄存器 `a2` 中, 因此输出时会显示寄存器 `a2` 的值。通过 `gdb` 调试可以验证这一点。

```
xv6 kernel is booting  
  
hart 1 starting  
hart 2 starting  
init: starting sh  
$ call  
x=3y=5301$
```

```
xv6-labs-2021 > kernel > C riscv.h  
366 typedef uint64 *pagetable_t; // 512 PTEs  
367  
368 // lab4: read the current frame pointer from s0 register  
369 static inline uint64 r_fp() {  
370     uint64 x;  
371     asm volatile("mv %0, s0" : "=r" (x) );  
372     return x;  
373 }
```

```
137 // lab4: print the return address  
138 void backtrace() {  
139     uint64 fp = r_fp();  
140     uint64 top = PGROUNDUP(fp);  
141     uint64 bottom = PGROUNDDOWN(fp);  
142     for (; fp >= bottom && fp < top; fp = *((uint64 *) (fp - 16))) {  
143         printf("%p\n", *((uint64 *) (fp - 8)));  
144     }  
145 }
```

```
xv6-labs-2021 > kernel > C defs.h
77     int pipewrite(struct pipe, uint64, int),
78
79     // printf.c
80     void printf(char*, ...);
81     void panic(char*) __attribute__((noreturn));
82     void printfinit(void);
83 | void backtrace(); // lab4
84
```

```
xv6-labs-2021 > kernel > C sysproc.c
56     sys_sleep(void){
64         while(ticks - ticks0 < n){
69             sleep(&ticks, &tickslock);
70         }
71         release(&tickslock);
72 |     backtrace(); // lab4-2
73         return 0;
74     }
```

```
xv6-labs-2021 > kernel > C printf.c
110
117     void
118     panic(char *s)
119     {
120         pr.locking = 0;
121         printf("panic: ");
122         printf(s);
123         printf("\n");
124 |     backtrace(); // lab4-2
125         panicked = 1; // freeze uart output from other CPUs
126     for(;;)
127         ;
128 }
```

```
xv6 kernel is booting
```

```
hart 1 starting
```

```
hart 2 starting
```

```
init: starting sh
```

```
$ bttest
```

```
0x000000008000214a
```

```
0x0000000080001fac
```

```
0x0000000080001c96
```

```
$
```

```
canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-traps backtrace
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_TRAPS -DLAB_TRAPS -MD -mcmodel=medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector -fno-pie -no-pie -c -o kernel/printf.o kernel/printf.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel kernel/entry.o kernel/kalloc.o kernel/string.o kernel/main.o kernel/vm.o kernel/proc.o kernel/swtch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/sleeplock.o kernel/file.o kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/plic.o kernel/virtio_disk.o kernel/start.o kernel/console.o kernel/printf.o kernel/uart.o kernel/spinlock.o
riscv64-linux-gnu-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-linux-gnu-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kernel/kernel.sym
== Test backtrace test == backtrace test: OK (0.7s)
(Old xv6.out.backtracetest failure log removed)
```

```
xv6-labs-2021 > user > C user.h
```

```
23 char* sbrk(int);
24 int sleep(int);
25 int uptime(void);
26 int sigalarm(int ticks, void (*handler)()); // lab4-3
27 int sigreturn(void); // lab4-3
```

```
xv6-labs-2021 > user > 🐼 usys.pl
```

```
37 entry("sleep");
38 entry("uptime");
39 entry("sigalarm"); # lab4-3
40 entry("sigreturn"); # lab4-3
```

```
xv6-labs-2021 > kernel > C syscall.h
```

```
21 #define SYS_mkdir 20
22 #define SYS_close 21
23 #define SYS_sigalarm 22 // lab4
24 #define SYS_sigreturn 23 // lab4
```

```
xv6-labs-2021 > kernel > C syscall.c
```

```
105 extern uint64 sys_write(void);
106 extern uint64 sys_uptime(void);
107 extern uint64 sys_sigalarm(void); // lab4
108 extern uint64 sys_sigreturn(void); // lab4
```

```
xv6-labs-2021 > kernel > C syscall.c
```

```
131 [SYS_close] sys_close,
132 [SYS_sigalarm] sys_sigalarm, // lab4
133 [SYS_sigreturn] sys_sigreturn, // lab4
134 };
```

```
xv6-labs-2021 > kernel > C sysproc.c
```

```
115 }
116
117 // lab4
118 uint64 sys_sigreturn(void) {
119     struct proc* p = myproc();
120     // trapframecopy must have the copy of trapframe
121     if(p->trapframecopy != p->trapframe + 512) {
122         return -1;
123     }
124     memmove(p->trapframe, p->trapframecopy, sizeof(struct trapframe)); // restore the trapframe
125     p->passedticks = 0; // prevent re-entrant
126     p->trapframecopy = 0;
127     return 0;
128 }
```

```
xv6-labs-2021 > kernel > C proc.h
```

```
86 struct proc {
87     struct inode *cwd; // Current directory
88     char name[16]; // Process name (debugging)
89     int interval; // alarm interval - lab4-3
90     uint64 handler; // lab4: pointer to the handler function
91     int passedticks; // lab4: ticks have passed since the last call
92     struct trapframe* trapframecopy; // lab4: the copy of trapframe
93 };
113
```

xv6-labs-2021 > kernel > C sysproc.c

```
97 }
98
99 // lab4
100 uint64 sys_sigalarm(void) {
101     int interval;
102     uint64 handler;
103     struct proc *p;
104
105     if (argint(0, &interval) < 0 || argaddr(1, &handler) < 0 || interval < 0) {
106         return -1;
107     }
108     // lab4
109     p = myproc();
110     p->interval = interval;
111     p->handler = handler;
112     p->passedticks = 0;
113
114     return 0;
115 }
```

xv6-labs-2021 > kernel > C proc.c

```
119 found:
138 // Set up new context to start executing at forkret,
139 // which returns to user space.
140 memset(&p->context, 0, sizeof(p->context));
141 p->context.ra = (uint64)forkret;
142 p->context.sp = p->kstack + PGSIZE;
143
144 p->interval = 0; // lab4
145 p->handler = 0; // lab4
146 p->passedticks = 0; // lab4
147 p->trapframecopy = 0; // lab4
148
149 return p;
150 }
```

```
37 usertrap(void){
78 // lab4
79 if(which_dev == 2){ // timer interrupt
80     // increase the passed ticks
81     if(p->interval != 0 && ++p->passedticks == p->interval){
82         // trapframecopy use the page of trapframe
83         p->trapframecopy = p->trapframe + 512;
84         memmove(p->trapframecopy, p->trapframe, sizeof(struct trapframe)); // copy trapframe
85         p->trapframe->epc = p->handler; // execute handler() when return to user space
86     }
87 }
88 // give up the CPU if this is a timer interrupt.
89 if(which_dev == 2)
90     yield();
91
92 usertrapret();
93 }
```



```
xv6-labs-2021 > M Makefile
```

```
174     UPROGS=\
```

```
188     $U/_gdt.img \
```

```
189     $U/_wc\
```

```
190     $U/_zombie\
```

```
191     $U/_alarmtest\|
```

```
192
```

```
193
```

```
xv6 kernel is booting
```

```
hart 1 starting
```

```
hart 2 starting
```

```
init: starting sh
```

```
$ alarmtest
```

```
test0 start
```

```
.....alarm!
```

```
test0 passed
```

```
test1 start
```

```
....alarm!
```

```
....alarm!
```

```
....alarm!
```

```
...alarm!
```

```
....alarm!
```

```
.....alarm!
```

```
....alarm!
```

```
....alarm!
```

```
...alarm!
```

```
....alarm!
```

```
.test1 passed
```

```
test2 start
```

```
.....alarm!
```

```
test2 passed
```

```
$
```

```
0x0000000080001ff6
0x0000000080001ca6
0x0000000080002194
0x0000000080001ff6
0x0000000080001ca6
0x0000000080002194
0x0000000080001ff6
0x0000000080001ca6
0x0000000080002194
0x0000000080001ff6
0x0000000080001ca6
0x0000000080002194
0x0000000080001ff6
0x0000000080001ca6
0x0000000080002194
0x0000000080001ff6
0x0000000080001ca6
0x0000000080002194
0x0000000080001ff6
0x0000000080001ca6
0x0000000080002194
0x0000000080001ff6
0x0000000080001ca6
OK
test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
$
```

```
canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-traps alarmtest
```

```
== Test running alarmtest == (3.8s)
== Test  alarmtest: test0 ==
alarmtest: test0: OK
== Test  alarmtest: test1 ==
alarmtest: test1: OK
== Test  alarmtest: test2 ==
alarmtest: test2: OK
```

5. Lab5 Copy-on-Write Fork for xv6

```
xv6-labs-2021 > kernel > C vm.c
299 // returns 0 on success, -1 on failure.
300 // frees any allocated pages on failure.
301 int
302 uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
303 {
304     pte_t *pte;
305     uint64 pa, i;
306     uint flags;
307     //char *mem;
308
309     for(i = 0; i < sz; i += PGSIZE){
310         if((pte = walk(old, i, 0)) == 0)
311             panic("uvmcopy: pte should exist");
312         if((*pte & PTE_V) == 0)
313             panic("uvmcopy: page not present");
314         pa = PTE2PA(*pte);
315         *pte = *pte & ~(PTE_W); // lab6: 清除状态
316         *pte = *pte | PTE_COW; //置cow为1
317         flags = PTE_FLAGS(*pte);
318         //if((mem = kalloc()) == 0)
319             // goto err;
320         //memmove(mem, (char*)pa, PGSIZE); //lab6 :直接传入pa
321         if(mappages(new, i, PGSIZE, (uint64)pa, flags) != 0){
322             //kfree(mem); //直接goerror
323             goto err;
324         }
325         incr((void *)pa); //lab6
```

xv6-labs-2021 > kernel > C vm.c

```
446 // lab6
447 int is_cow_fault(pagetable_t pagetable, uint64 va){
448     va = PGROUNDDOWN(va);
449     pte_t *pte = walk(pagetable, va, 0);
450
451     if(pte == 0)
452         return 0;
453     if((*pte & PTE_V) == 0)
454         return 0;
455     if((*pte & PTE_U) == 0)
456         return 0;
457
458     if(*pte & PTE_COW){
459         return 1;
460     }
461     return 0;
462 }
463
464 // lab6
465 int cow_alloc(pagetable_t pagetable, uint64 va){
466     va = PGROUNDDOWN(va);
467     pte_t *pte = walk(pagetable, va, 0);
468     uint64 pa = PTE2PA(*pte);
469     int flag = PTE_FLAGS(*pte);
470
471     char *mem = kalloc();
472     if(mem==0){
473         return -1;
474     }
475 }
```

xv6-labs-2021 > kernel > C kalloc.c

```
21 struct {
22     struct spinlock lock;
23     struct run *freelist;
24     char *ref_page; //lab6
25     int page_cnt;    //lab6
26     char *end_;      //lab6
27 } kmem;
```

xv6-labs-2021 > kernel > C kalloc.c

```
37
38 void
39 kinit()
40 {
41     initlock(&kmem.lock, "kmem");
42     kmem.page_cnt = pagecnt(end, (void *) PHYSTOP); //lab6
43
44     //lab6:
45     kmem.ref_page = end;
46     for(int i = 0; i < kmem.page_cnt; i++){
47         kmem.ref_page[i] = 0;
48     }
49     kmem.end_ = kmem.ref_page + kmem.page_cnt;
50
51     freerange(kmem.end_, (void*)PHYSTOP);
52 }
53
```

xv6-labs-2021 > kernel > C kalloc.c

```
52     }
53
54     //lab6:对页表进行计数
55     int page_index(uint64 pa){
56         pa = PGROUNDDOWN(pa);
57         int res = (pa - (uint64) kmem.end_) / PGSIZE;
58         if(res < 0 || res >= kmem.page_cnt){
59             panic("page_index illegal");
60         }
61         return res;
62     }
63
64     //lab6:
65     void incr(void *pa){
66         int index = page_index((uint64) pa);
67         acquire(&kmem.lock);
68         kmem.ref_page[index]++;
69         release(&kmem.lock);
70     }
71
72     //lab6:
73     void desc(void *pa){
74         int index = page_index((uint64) pa);
75         acquire(&kmem.lock);
76         kmem.ref_page[index]--;
77         release(&kmem.lock);
78     }
```

```
xv6-labs-2021 > kernel > C kalloc.c
93 void
94 kfree(void *pa)
95 {
96     //lab6:释放操作
97     int index = page_index((uint64) pa);
98     if(kmem.ref_page[index] > 1){
99         desc(pa);
100         return;
101     }
102
103     if(kmem.ref_page[index] == 1){
104         desc(pa);
105     }
106
107     struct run *r;
108
109     if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
110         panic("kfree");
111
112     // Fill with junk to catch dangling refs.
113     memset(pa, 1, PGSIZE);
114
115     r = (struct run*)pa;
```

```
xv6-labs-2021 > kernel > C kalloc.c
141 // Returns 0 if the memory cannot be allocated.
142 void *
143 kalloc(void)
144 {
145     struct run *r;
146
147     acquire(&kmem.lock);
148     r = kmem.freelist;
149     if(r)
150         kmem.freelist = r->next;
151     release(&kmem.lock);
152
153     if(r){
154         memset((char*)r, 5, PGSIZE); // fill with junk
155         //print_free_pages_cnt();
156         incr((void *)r); //lab6
157     }
158
159     return (void*)r;
160 }
```

Usertrap

```
xv6-labs-2021 > kernel > C trap.c
38 {
68 } else if((which_dev = devintr()) != 0){
69 // ok
70 } else if(r_scause()==15 || r_scause()==13){ //lab6
71     uint64 va = r_stval();
72     if(is_cow_fault(p->pagetable, va)){
73         if(cow_alloc(p->pagetable, va) < 0){
74             print("usertrap(): cow_alloc failed!")
75             p->killed = 1;
76         }
77     }
78     else{
79         printf("usertrap(): unexpected scause %p pid=%d\n", r_scause(), p->pid);
80         printf("          sepc=%p stval=%p\n", r_sepc(), r_stval());
81         p->killed = 1;
82     }
83 }
84 else {
85     printf("usertrap(): unexpected scause %p pid=%d\n", r_scause(), p->pid);
86     printf("          sepc=%p stval=%p\n", r_sepc(), r_stval());
87     p->killed = 1;
88 }
89 }
```

```
xv6-labs-2021 > kernel > C riscv.h
343 #define PTE_W (1L << 2)
344 #define PTE_X (1L << 3)
345 #define PTE_U (1L << 4) // 1 -> user can access
346 #define PTE_COW (1L << 8) // lab6: 8-> cow page
347
```

```
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
ALL COW TESTS PASSED
$
```


6. Lab6 Multithreading

6.1. 实验目的

本实验旨在深入理解和实现多线程机制，尤其是用户模式下的多线程切换。通过实现线程的创建、调度和切换，熟悉线程上下文的保存与恢复过程，并掌握多线程环境下的数据一致性问题及其解决方案。

6.2. 实验内容

本实验分为两个部分：

用户模式多线程的实现：通过模拟多线程的创建、调度和上下文切换，掌握基本的多线程操作。重点实现 `thread_create()`、`thread_schedule()` 和 `thread_switch()` 三个函数。

多线程环境下的数据一致性保护：针对线程不安全的哈希表进行并发访问测试，并通过引入互斥锁来解决数据丢失问题。重点理解并实现如何在多线程环境下保护共享数据结构，确保数据的一致性和正确性。

6.3. 实验步骤

Uthread: switching between threads

1、设置线程上下文结构体：

为了模拟多线程的切换，需要保存线程的上下文信息。首先定义一个结构体 `struct ctx`，用于保存线程切换时需要保存的寄存器信息。

```
xv6-labs-2021 > user > C uthread.c
12
13
14 // lab7: Saved registers for thread context switches.
15 struct ctx {
16     uint64 ra;
17     uint64 sp;
18
19     // callee-saved
20     uint64 s0;
21     uint64 s1;
22     uint64 s2;
23     uint64 s3;
24     uint64 s4;
25     uint64 s5;
26     uint64 s6;
27     uint64 s7;
28     uint64 s8;
29     uint64 s9;
30     uint64 s10;
31     uint64 s11;
32 };
33
```

将该结构体与每个线程结构体 `struct thread` 关联，以便在线程切换时可以访问并保存该线程的上下文。

2、在线程结构体中添加上下文字段：

在 `struct thread` 中添加一个 `context` 字段，用于保存该线程的上下文信息，包括程序计数器、栈指针以及其他需要保存的寄存器。

```
xv6-labs-2021 > user > C uthread.c
34 struct thread {
35     char    stack[STACK_SIZE]; /* the thread's stack */
36     int     state;             /* FREE, RUNNING, RUNNABLE */
37     struct ctx context;        // lab7-1: thread内容
38 };
39 struct thread all_thread[MAX_THREAD];
40 struct thread *current_thread;
41 extern void thread_switch(struct ctx *old, struct ctx *new); // lab7-1
42
```

3、实现 thread_create()函数:

在该函数中, 首先从线程数组中找到一个未初始化的线程, 将其状态设置为 RUNNABLE。将传递的函数指针和栈指针保存到线程的上下文中, 以确保线程在调度时能够正确执行指定的函数, 并使用自己的栈。

```
xv6-labs-2021 > user > C uthread.c
91 void
92 thread_create(void (*func)())
93 {
94     struct thread *t;
95
96     for (t = all_thread; t < all_thread + MAX_THREAD; t++) {
97         if (t->state == FREE) break;
98     }
99     t->state = RUNNABLE;
100     // YOUR CODE HERE
101     //lab7-1: 初始化
102     t->context.ra = (uint64) func;
103     t->context.sp = (uint64) t->stack + STACK_SIZE;
104 }
105
```

4、实现 thread_schedule()函数:

该函数用于从线程数组中寻找下一个可运行的线程并进行调度。如果找到下一个可运行的线程且当前线程不同, 则调用 thread_switch()进行线程切换。

```
xv6-labs-2021 > user > C uthread.c
57 {
78     if (current_thread != next_thread) { /* switch threads? */
79         t = current_thread;
80         current_thread = next_thread;
81         /* YOUR CODE HERE
82          * Invoke thread_switch to switch from t to next_thread:
83          * thread_switch(??, ??);
84          */
85         thread_switch(&t->context, &current_thread->context); // lab7-1: switch thread
86     } else
87         next_thread = 0;
88 }
89
```

5、实现 thread_switch()函数:

该函数使用汇编语言实现, 用于保存当前线程的上下文并恢复新线程的上下文, 实现线程切换。

```

xv6-labs-2021 > user > asm uthread_switch.S
7
8     .globl thread_switch
9     thread_switch:
10    /* YOUR CODE HERE */
11    # lab7 与switch.s中的原理相同
12    sd ra, 0(a0)
13    sd sp, 8(a0)
14    sd s0, 16(a0)
15    sd s1, 24(a0)
16    sd s2, 32(a0)
17    sd s3, 40(a0)
18    sd s4, 48(a0)
19    sd s5, 56(a0)
20    sd s6, 64(a0)
21    sd s7, 72(a0)
22    sd s8, 80(a0)
23    sd s9, 88(a0)
24    sd s10, 96(a0)
25    sd s11, 104(a0)
26
27    ld ra, 0(a1)
28    ld sp, 8(a1)
29    ld s0, 16(a1)
30    ld s1, 24(a1)

```

6、测试与验证：

在 xv6 中运行 uthread。

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  1

thread_c 98
thread_a 98
thread_b 98
thread_c 99
thread_a 99
thread_b 99
thread_c: exit after 100
thread_a: exit after 100
thread_b: exit after 100
thread_schedule: no runnable threads
$ 

```

./grade-lab-thread uthread 单项测试。

```

canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-thread uthread
make: 'kernel/kernel' is up to date.
== Test uthread == uthread: OK (0.9s)
(Old xv6.out.uthread failure log removed)

```

Using threads

首先进行预处理：（输入命令 ./ph 1 以及 ./ph 2）

```

● darli00@Darli0:~/xv6_labs/xv6-labs-2021$ make ph
gcc -o ph -g -O2 -DSOL_THREAD -DLAB_THREAD notxv6/ph.c -pthread
● darli00@Darli0:~/xv6_labs/xv6-labs-2021$ ./ph 1
100000 puts, 4.782 seconds, 20912 puts/second
0: 0 keys missing
100000 gets, 4.838 seconds, 20669 gets/second
● darli00@Darli0:~/xv6_labs/xv6-labs-2021$ ./ph 2
100000 puts, 2.168 seconds, 46125 puts/second
0: 16850 keys missing
1: 16850 keys missing
200000 gets, 5.132 seconds, 38975 gets/second
○ darli00@Darli0:~/xv6_labs/xv6-labs-2021$

```

1、定义互斥锁数组：

针对哈希表的每个桶(bucket)定义一个互斥锁，以便在访问桶中的链表时能够实现线程安全。

```

xv6-labs-2021 > notxv6 > C ph.c
18  int nthread = 1;
19  pthread_mutex_t locks[NBUCKET]; // lab7: 定义互斥锁数组
20

```

2、初始化互斥锁：

在 main()函数中，初始化哈希表中每个桶对应的互斥锁，以确保在并发访问时能够正确加锁。

```

xv6-labs-2021 > notxv6 > C ph.c
105  {
121  }
122
123  //lab7 初始化互斥锁
124  for(int i = 0; i < NBUCKET; ++i) {
125  |   pthread_mutex_init(&locks[i], NULL);
126  }
127

```

3、在 put()函数中实现加锁：

对于 put()函数，在操作哈希表的链表之前加锁，并在操作完成后解锁。这样可以避免多个线程同时访问同一桶中的链表时造成的数据丢失。（记得可以修改下面的宏定义，可以提高线程效率）

```

xv6-labs-2021 > notxv6 > C ph.c
7
8  #define NBUCKET 7 //lab7 增大该值
9  #define NKEYS 100000

```

```

xv6-labs-2021 > notxv6 > C ph.c
39 static
40 void put(int key, int value)
41 {
42     int i = key % NBUCKET;
43
44     // is the key already present?
45     struct entry *e = 0;
46     for (e = table[i]; e != 0; e = e->next) {
47         if (e->key == key)
48             break;
49     }
50     if(e){
51         // update the existing key.
52         e->value = value;
53     } else {
54         pthread_mutex_lock(&locks[i]);    // lab7: lock
55         // the new is new.
56         insert(key, value, &table[i], table[i]);
57         pthread_mutex_unlock(&locks[i]); // lab7: unlock
58     }

```

4、测试与验证：

./grade-lab-thread ph_safe 单项测试:

./grade-lab-thread ph_fast 单项测试:

```

canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-thread ph_safe
make: 'kernel/kernel' is up to date.
== Test ph_safe == gcc -o ph -g -O2 -DSOL_THREAD -DLAB_THREAD notxv6/ph.c -pthread
ph_safe: OK (6.7s)
canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-thread ph_fast
make: 'kernel/kernel' is up to date.
== Test ph_fast == make: 'ph' is up to date.
ph_fast: OK (12.6s)

```

Barrier

修改 barrier()函数:

```
xv6-labs-2021 > notxv6 > C barrier.c
24
25 static void
26 barrier()
27 {
28     // YOUR CODE HERE
29     //
30     // Block until all threads have called barrier() and
31     // then increment bstate.round.
32     // lab7
33     pthread_mutex_lock(&bstate.barrier_mutex);
34     // judge whether all threads reach the barrier
35     if(++bstate.nthread != nthread) { // not all threads reach
36         pthread_cond_wait(&bstate.barrier_cond, &bstate.barrier_mutex); // wait other threads
37     } else { // all threads reach
38         bstate.nthread = 0; // reset nthread
39         ++bstate.round; // increase round
40         pthread_cond_broadcast(&bstate.barrier_cond); // wake up all sleeping threads
41     }
42     pthread_mutex_unlock(&bstate.barrier_mutex);
43
44 }
```

测试：

输入./grade-lab-thread barrier

```
canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-thread barrier
make: 'kernel/kernel' is up to date.
== Test barrier == gcc -o barrier -g -O2 -DSOL_THREAD -DLAB_THREAD notxv6/barrier.c -pthread
barrier: OK (3.4s)
```

6.4. 实验心得

通过本次实验，我深入理解了用户模式下多线程的实现细节，特别是在线程创建、调度和切换过程中，如何正确保存和恢复线程的上下文。实验中遇到的挑战主要集中在如何保证线程切换时的正确性，以及如何在多线程环境下保护共享数据的安全性。

此外，实验二中对哈希表的线程安全保护使我更加理解了多线程编程中数据一致性的重要性。通过引入互斥锁，有效解决了多线程环境下的数据丢失问题，增强了程序的健壮性。

本实验不仅加深了我对多线程机制的理解，还使我意识到在实际编程中，如何通过合适的同步机制保护共享数据，确保程序的正确性和性能。这些经验和技巧将在未来的多线程编程中发挥重要作用。

7. Lab7 Networking

7.1. 实验目的

本实验旨在深入理解网卡驱动程序与 E1000 网卡硬件之间的交互逻辑，并通过实现关键函数来完成数据包的发送和接收。具体而言，实验要求实现 `e1000_transmit()` 和 `e1000_recv()` 函数，使得网卡驱动能够发送和接收数据包，从而实现网络通信的基本功能。

7.3. 实验内容

实现 `e1000_transmit()` 函数，该函数负责将要发送的数据帧放入网卡的发送队列，并通过网卡硬件完成数据的实际发送。

实现 `e1000_recv()` 函数，该函数从网卡的接收队列中提取接收到的数据帧，并将其传递给操作系统的网络栈进行进一步处理。

7.3. 实验步骤

Your Job

1、初始化发送队列

在 `e1000_init()` 函数中，初始化发送队列 `tx_ring`，包括描述符队列和缓冲区指针队列。同时，初始化网卡寄存器中相关的队列首尾指针。

2、实现 `e1000_transmit()` 函数

```
xv6-labs-2021 > kernel > C e1000.c
97 {
98 //
99 // Your code here.
100 //
101 // the mbuf contains an ethernet frame; program it into
102 // the TX descriptor ring so that the e1000 sends it. Stash
103 // a pointer so that it can be freed after sending.
104 // lab 7
105 uint32 tail;
106 struct tx_desc *desc;
107
108 acquire(&e1000_lock);
109 tail = regs[E1000_TDT];
110 desc = &tx_ring[tail];
111 // check if the ring is overflowing
112 if ((desc->status & E1000_TXD_STAT_DD) == 0) {
113     release(&e1000_lock);
114     return -1;
115 }
116 // free the last mbuf that was transmitted
117 if (tx_mbufs[tail]) {
118     mbuf_free(tx_mbufs[tail]);
119 }
120 // fill in the descriptor
121 desc->addr = (uint64) m->head;
122 desc->length = m->len;
123 desc->cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS;
124 tx_mbufs[tail] = m;
125
126 // a barrier to prevent reorder of instructions
127 __sync_synchronize();
128 regs[E1000_TDT] = (tail + 1) % TX_RING_SIZE;
129 release(&e1000_lock);
130 }
```

读取网卡寄存器中的发送尾指针 `TDT`，获取当前可以写入描述符的位置索引。

检查尾指针指向的描述符的 `status` 字段，确保描述符的 `DD` 标志位已被硬件设置，表示数据已被传输完毕。

如果上次发送的数据帧对应的缓冲区还未释放，则调用 `mbuffree()` 进行释放。

更新尾指针指向的描述符的 `addr` 字段，指向当前要发送的数据帧缓冲区的头部。同时，更新 `length` 字段以记录数据帧的长度。

设置描述符的 `cmd` 字段，包含 `EOP` 和 `RS` 标志位，分别表示数据帧结束和请求状态报告。

将当前的数据帧缓冲区指针存储在 `tx_mbufs` 中，以便后续释放。

设置内存屏障，确保描述符的更新在移动尾指针之前完成。

更新网卡寄存器中的发送尾指针 TDT，使其指向下一个描述符位置。

Hints

1、初始化接收队列

在 `e1000_init()` 函数中，初始化接收队列 `rx_ring`，包括描述符队列和缓冲区指针队列。同时，初始化网卡寄存器中相关的队列首尾指针。

2、实现 `e1000_recv()` 函数

```
xv6-labs-2021 > kernel > C e1000.c
134 static void
135 e1000_recv(void)
136 {
137     //
138     // Your code here.
139     //
140     // Check for packets that have arrived from the e1000
141     // Create and deliver an mbuf for each packet (using net_rx()).
142     // lab 7
143     int tail = (regs[E1000_RDT] + 1) % RX_RING_SIZE;
144     struct rx_desc *desc = &rx_ring[tail];
145
146     while ((desc->status & E1000_RXD_STAT_DD)) {
147         if(desc->length > MBUF_SIZE) {
148             panic("e1000 len");
149         }
150         // update the length reported in the descriptor.
151         rx_mbufs[tail]->len = desc->length;
152         // deliver the mbuf to the network stack
153         net_rx(rx_mbufs[tail]);
154         // allocate a new mbuf replace the one given to net_rx()
155         rx_mbufs[tail] = mbufalloc(0);
156         if (!rx_mbufs[tail]) {
157             panic("e1000 no mbufs");
158         }
159         desc->addr = (uint64) rx_mbufs[tail]->head;
160         desc->status = 0;
161
162         tail = (tail + 1) % RX_RING_SIZE;
163         desc = &rx_ring[tail];
164     }
```

读取网卡寄存器中的接收尾指针 RDT 并加 1 取余，获取当前未处理的数据帧所在的描述符位置索引。

检查描述符的 `status` 字段中的 DD 标志位，确保数据帧已被硬件接收完毕。

将接收缓冲区中的数据帧长度记录在描述符的 `length` 字段，并调用 `net_rx()` 函数将数据传递给网络栈进行处理。

为接收队列分配一个新的缓冲区，并更新描述符的 `addr` 字段，指向新缓冲区。

清空描述符的 `status` 字段，准备好描述符供硬件接收新数据。

循环处理队列中的所有可解封装的数据帧，直至当前描述符的 DD 标志位未被设置。

更新网卡寄存器中的接收尾指针 RDT，使其指向最后一个处理完的数据帧。

3、测试

在 `xv6` 目录下执行 `make server` 启动服务端。


```
canyon@Rikka:~/xv6-labs-2021$ make server
python3 server.py 26099
listening on localhost port 26099
a message from xv6!
a message from xv6!
a message from xv6!
a message from xv6!
a message from xv6!
```

然后在另一个终端执行 `make qemu` 启动 `xv6`，然后执行 `nettests` 命令进行测试：

```
hart 2 starting
init: starting sh
$ nettests
nettests running on port 26099
testing ping: OK
testing single-process pings: OK
testing multi-process pings: OK
testing DNS
DNS arecord for pdos.csail.mit.edu. is 128.52.129.126
DNS OK
all tests passed.
```

然后在终端执行 `tcpdump -XXnr packets.pcap`，可以查看捕获的报文：

```
0x0000: ffff ffff ffff 5254 0012 3456 0800 4500 .....RT..4V..E.
0x0010: 002f 0000 0000 6411 3eae 0a00 020f 0a00 ./....d.>.....
0x0020: 0202 07d0 65f3 001b 0000 6120 6d65 7373 ....e.....a.mess
0x0030: 6167 6520 6672 6f6d 2078 7636 21 age.from.xv6!
09:35:18.151367 IP 10.0.2.2.26099 > 10.0.2.15.2000: UDP, length 17
0x0000: 5254 0012 3456 5255 0a00 0202 0800 4500 RT..4VRU.....E.
0x0010: 002d 0009 0000 4011 62a7 0a00 0202 0a00 -....@.b.....
0x0020: 020f 65f3 07d0 0019 3216 7468 6973 2069 ..e.....2.this.i
0x0030: 7320 7468 6520 686f 7374 2100 s.the.host!.
09:35:18.151661 IP 10.0.2.15.2000 > 10.0.2.2.26099: UDP, length 19
0x0000: ffff ffff ffff 5254 0012 3456 0800 4500 .....RT..4V..E.
0x0010: 002f 0000 0000 6411 3eae 0a00 020f 0a00 ./....d.>.....
0x0020: 0202 07d0 65f3 001b 0000 6120 6d65 7373 ....e.....a.mess
0x0030: 6167 6520 6672 6f6d 2078 7636 21 age.from.xv6!
09:35:18.151844 IP 10.0.2.2.26099 > 10.0.2.15.2000: UDP, length 17
0x0000: 5254 0012 3456 5255 0a00 0202 0800 4500 RT..4VRU.....E.
0x0010: 002d 000a 0000 4011 62a6 0a00 0202 0a00 -....@.b.....
0x0020: 020f 65f3 07d0 0019 3216 7468 6973 2069 ..e.....2.this.i
0x0030: 7320 7468 6520 686f 7374 2100 s.the.host!.
```

`make grade` 测试：

```
== Test running nettests ==
$ make qemu-gdb
(3.0s)
== Test nettest: ping ==
nettest: ping: OK
== Test nettest: single process ==
nettest: single process: OK
== Test nettest: multi-process ==
nettest: multi-process: OK
== Test nettest: DNS ==
nettest: DNS: OK
```

7.4. 实验心得

本次实验通过实现 `E1000` 网卡驱动中的关键函数，进一步加深了对网卡硬件与驱动程序交互机制的理解。在实现 `e1000_transmit()` 和 `e1000_recv()` 函数的过程中，重点关注了

描述符队列的操作和网卡寄存器的配置, 体会到硬件与软件之间通过共享的数据结构进行交互的设计理念。此外, 通过对 DD 标志位和 RS 标志位的处理, 理解了如何确保数据的正确传输和接收。本实验不仅强化了网络编程的基础知识, 还提升了实际动手能力, 对于深入理解操作系统中的网络子系统大有裨益。

8. Lab8 Lock

8.1. 实验目的

通过修改 xv6 的内存分配器和缓冲区缓存机制, 减少锁的争用, 提升系统在多核环境下的性能。具体目标是:

重构内存分配器, 减少内存页分配和释放过程中的锁争用。

优化缓冲区缓存的锁争用问题, 采用哈希表结构来替代原有的双向链表结构, 以提高缓存管理效率。

8.2. 实验内容

实验分为两个部分:

内存分配器的优化: 通过将内存分配器中的单一锁和单一链表拆分为每个 CPU 独立的锁和链表, 从而减少锁的争用。

缓冲区缓存的优化: 将缓冲区缓存的管理由原来的双向链表改为基于哈希表的结构, 并对缓存块的管理算法进行优化, 以减少锁的争用和提高缓存命中率。

8.3. 实验步骤

Memory allocator

1、修改内存页 kmems 数组结构

将 kmem 结构体替换为 kmems 数组, 数组大小为 CPU 的核心数 NCPU。

```
xv6-labs-2021 > kernel > C kalloc.c
20
21 struct {
22     struct spinlock lock;
23     struct run *freelist;
24     char lockname[8]; // lab6: 保存锁名
25 } kmems[NCPU]; // lab8: 给每个CPU上锁
26
```

每个数组元素包含一个锁和一个空闲内存页链表, 以及一个用于记录锁名称的字段 lockname。

2、修改初始化函数 kinit()

在 kinit() 中, 使用循环初始化 kmems 数组中每个元素的锁, 并调用 freerange() 初始化物理页分配。

```

xv6-labs-2021 > kernel > C kalloc.c
26
27 void
28 kinit()
29 {
30     // lab8: 初始化kmem
31     int i;
32     for (i = 0; i < NCPU; ++i) {
33         snprintf(kmems[i].lockname, 8, "kmem %d", i);
34         initlock(&kmems[i].lock, kmems[i].lockname);
35     }
36     freerange(end, (void*)PHYSTOP);
37 }
38
39 void
40 freerange(void *pa_start, void *pa_end)
41 {
42     char *p;
43     p = (char*)PGROUNDUP((uint64)pa_start);
44     for(; p + PGSIZE <= (char*)pa_end; p += PGSIZE)
45         kfree(p);
46 }
47

```

使用 `snprintf()` 函数为每个锁生成唯一的名称，并将名称存储在 `lockname` 字段中。

3、修改 `kfree()` 函数

在 `kfree()` 中，使用 `cpuid()` 获取当前 CPU 核心的序号，将释放的物理页回收到对应的 `kmems` 链表中。

```

xv6-labs-2021 > kernel > C kalloc.c
52 // initializing the allocator; see kinit above.)
53 void
54 kfree(void *pa)
55 {
56     struct run *r;
57     int cpu_id; //lab8: cpuid
58
59     if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
60         panic("kfree");
61
62     // Fill with junk to catch dangling refs.
63     memset(pa, 1, PGSIZE);
64
65     r = (struct run*)pa;
66
67     // lab8: 获取core number
68     push_off();
69     cpu_id = cpuid();
70     pop_off();
71
72     // lab8: 释放cpu页
73     acquire(&kmems[cpu_id].lock);
74     r->next = kmems[cpu_id].freelist;
75     kmems[cpu_id].freelist = r;
76     release(&kmems[cpu_id].lock);
77 }

```

4、修改 `kalloc()` 函数

使用 `cpuid()` 获取当前 CPU 核心的序号，从对应的 `kmems` 链表中分配物理页。如果当前 CPU 的链表为空，调用 `steal()` 函数从其他 CPU 的链表中偷取部分空闲物理页。

```

xv6-labs-2021 > kernel > C kalloc.c
131 kalloc(void)
132 {
133     struct run *r;
134     // lab8: 获取当前CPU的ID
135     int c;
136     push_off();
137     c = cpuid();
138     pop_off();
139
140     // 从当前CPU的空闲内存页链表中获取一页内存
141     acquire(&kmems[c].lock);
142     r = kmems[c].freelist;
143     if (r)
144         kmems[c].freelist = r->next;
145     release(&kmems[c].lock);
146
147     // lab8-1: 如果当前CPU没有空闲页, 则从其他CPU“偷”一页
148     if (!r && (r = steal(c))) {
149         acquire(&kmems[c].lock);
150         kmems[c].freelist = r->next;
151         release(&kmems[c].lock);
152     }
153
154     // 如果成功分配到内存页, 将其填充为垃圾数据
155     if (r)
156         memset((char*)r, 5, PGSIZE); // 填充为垃圾数据
157
158     // 返回分配的内存页地址
159     return (void*)r;
160 }

```

5、编写偷取物理页函数 steal()

实现 steal() 函数, 当当前 CPU 的空闲物理页链表为空时, 从其他 CPU 的链表中偷取部分物理页。

```

xv6-labs-2021 > kernel > C kalloc.c
78
79 // lab8: 从其他 CPU 的空闲内存页链表中偷取一半的页
80 struct run *steal(int cpu_id) {
81     int i;
82     int c = cpu_id;
83     struct run *fast, *slow, *head;
84
85     // 检查当前 CPU ID 是否与传入的 CPU ID 相同
86     if (cpu_id != cpuid()) {
87         panic("steal");
88     }
89
90     // 遍历其他 CPU 的空闲内存页链表
91     for (i = 1; i < NCPU; ++i) {
92         if (++c == NCPU) {
93             c = 0; // 如果到达最后一个 CPU, 则从第一个 CPU 重新开始
94         }
95
96         // 获取目标 CPU 的内存池锁
97         acquire(&kmems[c].lock);
98
99         // 如果目标 CPU 的空闲内存页链表不为空
100         if (kmems[c].freelist) {
101             // 使用快慢指针法找到链表中间位置
102             slow = head = kmems[c].freelist;
103             fast = slow->next;
104             while (fast) {
105                 fast = fast->next;

```

通过遍历其他 CPU 的链表, 寻找非空链表, 并使用快慢指针算法将其分成两部分, 偷取一半的物理页。

6、测试

在 xv6 中执行 kallocstest, 输出如下, 可以看到对于每个 CPU 的物理页锁的争用情况相比之前有明显下降, acquire() 整体次数大幅减少, 同时 kmems 中的锁也不再是最具争用性的 5 个锁。测试 test1 和 test2 也均通过。

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ kallocstest
start test1
test1 results:
--- lock kmem/bcache stats
lock: kmem_0: #test-and-set 0 #acquire() 71421
lock: kmem_1: #test-and-set 0 #acquire() 186065
lock: kmem_2: #test-and-set 0 #acquire() 175535
lock: kmem_3: #test-and-set 0 #acquire() 2
lock: kmem_4: #test-and-set 0 #acquire() 2
lock: kmem_5: #test-and-set 0 #acquire() 2
lock: kmem_6: #test-and-set 0 #acquire() 2
lock: kmem_7: #test-and-set 0 #acquire() 2
lock: bcache: #test-and-set 0 #acquire() 1254
--- top 5 contended locks:
lock: proc: #test-and-set 132557 #acquire() 517786
lock: proc: #test-and-set 23148 #acquire() 517884
lock: proc: #test-and-set 21127 #acquire() 517732
lock: virtio_disk: #test-and-set 19074 #acquire() 114
lock: proc: #test-and-set 17014 #acquire() 517889
tot= 0
test1 OK
start test2
total free number of pages: 32499 (out of 32768)
.....
test2 OK
$
```

在 xv6 中执行 usertests sbrkmuch 进行测试:

```
$ usertests sbrkmuch
usertests starting
test sbrkmuch: OK
ALL TESTS PASSED
$
```

在 xv6 中执行 usertests 测试:

```
OK
test sbrkfail: usertrap(): unexpected scause 0x000000000000000d pid=6306
sepc=0x00000000000004fc stval=0x0000000000012000
OK
test sbrkarg: OK
test sbrklast: OK
test sbrk0000: OK
test validatetest: OK
test stacktest: usertrap(): unexpected scause 0x000000000000000d pid=6312
sepc=0x0000000000002376 stval=0x00000000000fb50
OK
test opentest: OK
test writetest: OK
test writebig: OK
test createtest: OK
test openiput: OK
test exitiput: OK
test iput: OK
test mem: OK
test pipe1: OK
test killstatus: OK
test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
```

./grade-lab-lock kalloc_test 单项测试:

```
canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-lock kalloc_test
make: 'kernel/kernel' is up to date.
== Test running kalloc_test == (35.2s)
== Test kalloc_test: test1 ==
kalloc_test: test1: OK
== Test kalloc_test: test2 ==
kalloc_test: test2: OK
== Test kalloc_test: sbrkmuch == kalloc_test: sbrkmuch: OK (5.0s)
```

Buffer cache

预处理: (输入 bcachetest)

```
hart 1 starting
hart 2 starting
init: starting sh
$ bcachetest
start test0
test0 results:
--- lock kmem/bcache stats
lock: kmem_0: #test-and-set 0 #acquire() 32918
lock: kmem_1: #test-and-set 0 #acquire() 50
lock: kmem_2: #test-and-set 0 #acquire() 72
lock: kmem_3: #test-and-set 0 #acquire() 1
lock: kmem_4: #test-and-set 0 #acquire() 1
lock: kmem_5: #test-and-set 0 #acquire() 1
lock: kmem_6: #test-and-set 0 #acquire() 1
lock: kmem_7: #test-and-set 0 #acquire() 1
lock: bcache: #test-and-set 41270 #acquire() 72500
--- top 5 contended locks:
lock: virtio_disk: #test-and-set 155886 #acquire() 1239
lock: proc: #test-and-set 137562 #acquire() 922276
lock: bcache: #test-and-set 41270 #acquire() 72500
lock: proc: #test-and-set 36942 #acquire() 901767
lock: proc: #test-and-set 33821 #acquire() 922258
tot= 41270
test0: FAIL
start test1
test1 OK
$
```

1、修改 buf 结构体

在 buf 结构体中移除双向链表的 prev 字段, 并添加 timestamp 字段用于记录缓存块的最后使用时间。

```
xv6-labs-2021 > kernel > C buf.h
1 struct buf {
2     int valid; // has data been read from disk?
3     int disk; // does disk "own" buf?
4     uint dev;
5     uint blockno;
6     struct sleeplock lock;
7     uint refcnt;
8     struct buf *next; // hash list
9     uchar data[BSIZE];
10    uint timestamp; // lab8: 最后一次使用buf的时间
11 };
12
```

2、初始化哈希表结构

初始化哈希表结构, 使用 bcache 中的全局锁分配缓存块, 并将缓存块插入哈希表的 bucket 中。

```

xv6-labs-2021 > kernel > C bio.c
24 #include "buf.h"
25
26 // lab8
27 #define NBUCKET 13 // the count of hash table's buckets
28 #define HASH(blockno) (blockno % NBUCKET)
29
30 extern uint ticks; // lab8
31
32 struct {
33     struct spinlock lock; // used for the buf alloc and size
34     struct buf buf[NBUF];
35     // lab8
36     int size; // record the count of used buf
37     struct buf buckets[NBUCKET]; // lab8
38     struct spinlock locks[NBUCKET]; // buckets' locks
39     struct spinlock hashlock; // the hash table's lock
40     // Linked list of all buffers, through prev/next.
41     // Sorted by how recently the buffer was used.
42     // head.next is most recent, head.prev is least.
43     struct buf head; // lab8: delete
44 } bcache;

```

针对缓存块的重新分配情况，使用全局锁确保哈希表的一致性。

3、修改 bget() 函数

在 bget() 函数中，通过哈希表来查找缓存块，并使用 bcache 的全局锁管理缓存块的初始分配。

```

xv6-labs-2021 > kernel > C bio.c
76 // Look through buffer cache for block on device dev.
77 // If not found, allocate a buffer.
78 // In either case, return locked buffer.
79 static struct buf*
80 bget(uint dev, uint blockno)
81 {
82     struct buf *b;
83     // lab8-2: 通过 HASH 函数计算 blockno 对应的桶索引
84     int idx = HASH(blockno);
85     struct buf *pre, *minb = 0, *minpre;
86     uint mintimestamp;
87     int i;
88
89     // 在对应的哈希桶中查找缓存的 buf
90     acquire(&bcache.locks[idx]); // lab8-2
91     for(b = bcache.buckets[idx].next; b; b = b->next){
92         if(b->dev == dev && b->blockno == blockno){
93             b->refcnt++; // 增加引用计数
94             release(&bcache.locks[idx]); // lab8-2
95             acquiresleep(&b->lock);
96             return b;
97         }
98     }
99
100     // 缓存未命中
101     // 检查是否有未使用的 buf - lab8-2
102     acquire(&bcache.lock);
103     if(bcache.size < NBUF) {
104         b = &bcache.buf[bcache.size++];

```

当缓存块未找到时，使用时间戳机制寻找最近未使用的块，并进行重用。

4、修改 brelse() 函数

修改 brelse() 以及相关函数，释放缓存块时根据当前时间更新其 timestamp，并重新插入到哈希表中。

xv6-labs-2021 > kernel > C bio.c

```
46 void
47 binit(void)
48 {
49     int i; //lab8
50     struct buf *b;
51
52     bcache.size = 0; // lab8
53
54     initlock(&bcache.lock, "bcache");
55
56     initlock(&bcache.hashlock, "bcache_hash"); // lab8: init hash lock
57
58     // lab8: 初始化所有bucket locks
59     for(i = 0; i < NBUCKET; ++i) {
60         initlock(&bcache.locks[i], "bcache_bucket");
61     }
62
63     // lab8 delete
64     // Create linked list of buffers
65     //bcache.head.prev = &bcache.head;
66     //bcache.head.next = &bcache.head;
67     for(b = bcache.buf; b < bcache.buf+NBUF; b++){
68         //b->next = bcache.head.next;
69         //b->prev = &bcache.head;
70         initsleeplock(&b->lock, "buffer");
71         //bcache.head.next->prev = b;
72         //bcache.head.next = b;
73     }
74 }
```

xv6-labs-2021 > kernel > C bio.c

```
135 // Release a locked buffer.
136 // Move to the head of the most-recently-used list.
137 void
138 brelse(struct buf *b)
139 {
140     int idx; //lab8
141     if(!holdingsleep(&b->lock))
142         panic("brelse");
143
144     releasesleep(&b->lock);
145
146     // lab8: change the lock
147     idx = HASH(b->blockno);
148
149     acquire(&bcache.lock);
150     b->refcnt--;
151     if (b->refcnt == 0) {
152         // no one is waiting for it.
153         //lab8: delete
154         //b->next->prev = b->prev;
155         //b->prev->next = b->next;
156         //b->next = bcache.head.next;
157         //b->prev = &bcache.head;
158         //bcache.head.next->prev = b;
159         //bcache.head.next = b;
160         b->timestamp = ticks; //lab8
161     }
162
163     release(&bcache.lock);
164 }
```



```

xv6-labs-2021 > kernel > C bio.c
164     }
165
166     void
167     bpin(struct buf *b) {
168         // lab8: change the bpin
169         int idx = HASH(b->blockno);
170         acquire(&bcache.locks[idx]);
171         b->refcnt++;
172         release(&bcache.locks[idx]);
173     }
174
175     void
176     bunpin(struct buf *b) {
177         // lab8: change the bunpin
178         int idx = HASH(b->blockno);
179         acquire(&bcache.locks[idx]);
180         b->refcnt--;
181         release(&bcache.locks[idx]);
182     }

```

5、测试

在 xv6 中运行 bcachetest, 输出如下, 可以看到 bcache 相关锁的争用情况大幅下降, acquire() 整体次数大幅减少, 同时 bcache 中的锁也不再是最具争用性的 5 个锁。测试 test0 和 test1 也均通过。

```

xv6-labs-2021 > kernel > C bio.c
198     {
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1

start test0
test0 results:
--- lock kmem/bcache stats
lock: kmem_0: #test-and-set 0 #acquire() 32906
lock: kmem_1: #test-and-set 0 #acquire() 76
lock: kmem_2: #test-and-set 0 #acquire() 56
lock: kmem_3: #test-and-set 0 #acquire() 2
lock: kmem_4: #test-and-set 0 #acquire() 2
lock: kmem_5: #test-and-set 0 #acquire() 2
lock: kmem_6: #test-and-set 0 #acquire() 2
lock: kmem_7: #test-and-set 0 #acquire() 2
lock: bcache: #test-and-set 0 #acquire() 85
lock: bcache_hash: #test-and-set 0 #acquire() 55
lock: bcache_bucket: #test-and-set 0 #acquire() 4122
lock: bcache_bucket: #test-and-set 0 #acquire() 4134
lock: bcache_bucket: #test-and-set 0 #acquire() 2272
lock: bcache_bucket: #test-and-set 0 #acquire() 4278
lock: bcache_bucket: #test-and-set 0 #acquire() 2266
lock: bcache_bucket: #test-and-set 0 #acquire() 4261
lock: bcache_bucket: #test-and-set 0 #acquire() 4741
lock: bcache_bucket: #test-and-set 25 #acquire() 9010
lock: bcache_bucket: #test-and-set 0 #acquire() 6179
lock: bcache_bucket: #test-and-set 0 #acquire() 6179
lock: bcache_bucket: #test-and-set 0 #acquire() 6179
lock: bcache_bucket: #test-and-set 0 #acquire() 6181
lock: bcache_bucket: #test-and-set 0 #acquire() 6201
--- top 5 contended locks:
lock: virtio_disk: #test-and-set 151910 #acquire() 1188
lock: proc: #test-and-set 51272 #acquire() 1836416
lock: proc: #test-and-set 48304 #acquire() 1816467
lock: proc: #test-and-set 47098 #acquire() 1816462
lock: proc: #test-and-set 43379 #acquire() 1816469
tot= 25
test0: OK
start test1
test1 OK
$

```

在 xv6 中执行 usertests 测试:

```
xv6-labs-2021 > kernel > C bio.c
19R {
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1
sepc=0x00000000000002158 stval=0x000000000001c3a90
usertrap(): unexpected scause 0x000000000000000d pid=6292
sepc=0x00000000000002158 stval=0x000000000001cfde0
usertrap(): unexpected scause 0x000000000000000d pid=6293
sepc=0x00000000000002158 stval=0x000000000001dc130
OK
test sbrkfail: usertrap(): unexpected scause 0x000000000000000d pid=6305
sepc=0x000000000000044fc stval=0x0000000000012000
OK
test sbrkarg: OK
test sbrklast: OK
test sbrk000: OK
test validate: OK
test stacktest: usertrap(): unexpected scause 0x000000000000000d pid=6311
sepc=0x00000000000002376 stval=0x000000000000fb50
OK
test opentest: OK
test writetest: OK
test writebig: OK
test createtest: OK
test openinput: OK
test exitinput: OK
test iput: OK
test mem: OK
test pipe1: OK
test killstatus: OK
test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
$
```

make grade 测试:

```
$ make qemu-gdb
(53.0s)
== Test kalloc: test1 ==
kalloc: test1: OK
== Test kalloc: test2 ==
kalloc: test2: OK
== Test kalloc: sbrkmuch ==
$ make qemu-gdb
kalloc: sbrkmuch: OK (6.9s)
== Test running bcachetest ==
$ make qemu-gdb
(7.1s)
== Test bcachetest: test0 ==
bcachetest: test0: OK
== Test bcachetest: test1 ==
bcachetest: test1: OK
== Test usertests ==
$ make qemu-gdb
usertests: OK (95.0s)
```

8.3. 实验心得

通过本次实验，我深刻体会到了在多核环境下如何优化锁的使用以减少争用，从而提升系统的性能。具体来说，将共享数据结构分解为每个 CPU 独立的部分，配合合理的锁机制，可以大幅减少锁的争用。与此同时，在管理共享资源时，需要特别注意避免可能导致死锁的情况，并仔细设计数据结构和算法，以确保在并发环境中的一致性和效率。这些经验对于以后处理多核并发编程和系统优化都有非常重要的参考价值。

9. Lab9 File System

9.1. 实验目的

本次实验的目的是在 xv6 操作系统中实现对文件系统的扩展，主要包括实现二级间接块索引的 inode 结构，并修改相关函数以支持和管理更大文件。此外，实验还涉及实现符号链接的系统调用，并调整文件系统的相关操作以正确处理符号链接。

9.2. 实验内容

实验主要包含以下两部分内容：

实现二级间接块索引：通过扩展 xv6 文件系统的 inode 结构，支持更大文件的存储。具体实现包括修改 inode 结构和相关函数，如 bmap() 和 itrunc()，以适配新的存储结构。

实现符号链接：添加符号链接（软链接）的系统调用 symlink，并修改 open 系统调用以处理符号链接的情况，包括递归查找目标文件和避免循环引用。

9.3. 实验步骤

Large file

1、修改 inode 结构：

将 kernel/fs.h 中的直接块号宏定义 NDIRECT 修改为 11，并调整 inode 相关结构体的块号数组，将数组大小设置为 NDIRECT+2，以适应二级间接块索引的需求。

```
xv6-labs-2021 > kernel > C fs.h
25 #define FSMAGIC 0x10203040
26
27 #define NDIRECT 11 //lab9 改为11
28 #define NINDIRECT (BSIZE / sizeof(uint))
29
30 #define NDOUBLYINDIRECT (NINDIRECT * NINDIRECT) // lab9
31 #define MAXFILE (NDIRECT + NINDIRECT + NDOUBLYINDIRECT) // lab9
32
33
34 // On-disk inode structure
35 struct dinode {
36     short type; // File type
37     short major; // Major device number (T_DEVICE only)
38     short minor; // Minor device number (T_DEVICE only)
39     short nlink; // Number of links to inode in file system
40     uint size; // Size of file (bytes)
41     uint addrs[NDIRECT+2]; // Data block addresses lab9: 改为+2
42 };
43
```

在 kernel/fs.h 中添加宏定义 NDOUBLYINDIRECT，表示二级间接块号的总数。

```
xv6-labs-2021 > kernel > C fs.h
61     };
62
63     // lab9: the max depth of symlinks
64     #define NSYMLINK 10
```

```
xv6-labs-2021 > kernel > C file.h
16 // in-memory copy of an inode
17 struct inode {
18     uint dev;           // Device number
19     uint inum;          // Inode number
20     int ref;            // Reference count
21     struct sleeplock lock; // protects everything below here
22     int valid;          // inode has been read from disk?
23
24     short type;         // copy of disk inode
25     short major;
26     short minor;
27     short nlink;
28     uint size;
29     uint addrs[NDIRECT+2]; // lab9: 改为+2
30 };
```

2、修改 bmap() 函数：

修改 kernel/fs.c 中的 bmap() 函数，以支持二级间接块索引。当请求的块号大于直接块号和一级间接块号时，函数会递归地处理二级间接块索引，分配或获取相应的磁盘块。

```
378 bmap(struct inode *ip, uint bn){
402
403     // lab9: 处理双重间接块
404
405     // 减去一级间接块的数量，bn 现在表示双重间接块中的块号
406     bn -= NINDIRECT;
407
408     // 如果 bn 位于双重间接块范围内
409     if(bn < NDOUBLYINDIRECT) {
410         // 获取双重间接块的地址
411         if((addr = ip->addrs[NDIRECT + 1]) == 0) {
412             // 如果双重间接块还没有分配，则分配一个新的块
413             ip->addrs[NDIRECT + 1] = addr = balloc(ip->dev);
414         }
415         // 读取双重间接块
416         bp = bread(ip->dev, addr);
417         a = (uint*)bp->data;
418
419         // 获取对应的一级间接块地址
420         if((addr = a[bn / NINDIRECT]) == 0) {
421             // 如果一级间接块还没有分配，则分配一个新的块
422             a[bn / NINDIRECT] = addr = balloc(ip->dev);
423             log_write(bp); // 写入日志，确保操作的原子性
424         }
425         brelse(bp); // 释放双重间接块的缓存
426
427         // 读取一级间接块
428         bp = bread(ip->dev, addr);
429         a = (uint*)bp->data;
```

3、修改 itrunc() 函数：

在 kernel/fs.c 中修改 itrunc() 函数，以释放二级间接块的磁盘块。通过两重循环，分别遍历二级间接块及其内部的一级间接块，并释放对应的磁盘块。

4、修改 MAXFILE 定义：

更新 kernel/fs.h 中的文件最大大小宏定义 MAXFILE，以反映二级间接块支持的最大文件大小。

5、在 xv6 中执行 bigfile 测试：

在 xv6 中执行 bigfile:

```

init: starting sh
$ bigfile
.....
.....
.....
wrote 65803 blocks
bigfile done; ok
$

```

在 xv6 中执行 usertests:

```

usertrap(): unexpected scause 0x000000000000000d pid=6234
      sepc=0x0000000000000215a stval=0x00000000000001c130
OK
test sbrkfail: usertrap(): unexpected scause 0x000000000000000d pid=6242
      sepc=0x000000000000044fe stval=0x000000000000012000
OK
test sbrkarg: OK
test sbrklast: OK
test sbrk8000: OK
test validate: OK
test stacktest: usertrap(): unexpected scause 0x000000000000000d pid=6248
      sepc=0x00000000000002378 stval=0x0000000000000fb50
OK
test opentest: OK
test writetest: OK
test writebig: OK
test createtest: OK
test openinput: OK
test exitinput: OK
test input: OK
test mem: OK
test pipe1: OK
test killstatus: OK
test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
$

```

```

canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-fs bigfile
make: 'kernel/kernel' is up to date.
== Test running bigfile == running bigfile: OK (55.1s)

```

Symbolic links

1、实现符号链接系统调用:

在 kernel/syscall.h、kernel/syscall.c、user/usys.pl 和 user/user.h 中添加 symlink 系统调用的声明。

```

xv6-labs-2021 > kernel > C syscall.h
22 #define SYS_close 21
23 #define SYS_symlink 22 // lab9

```

```

xv6-labs-2021 > kernel > C syscall.c
105 extern uint64 sys_write(void);
106 extern uint64 sys_uptime(void);
107 extern uint64 sys_symlink(void); // lab9

```

```

xv6-labs-2021 > kernel > C syscall.c
128 [SYS_link] sys_link,
129 [SYS_mkdir] sys_mkdir,
130 [SYS_close] sys_close,
131 [SYS_symlink] sys_symlink, //lab9
132 };

```

```
xv6-labs-2021 > user >  usys.pl
37 entry("sleep");
38 entry("uptime");
39 entry("symlink"); # lab9
```

```
xv6-labs-2021 > user >  user.h
24 int sleep(int);
25 int uptime(void);
26 int symlink(char *target, char *path); // lab9
```

```
xv6-labs-2021 > kernel >  stat.h
1 #define T_DIR 1 // Directory
2 #define T_FILE 2 // File
3 #define T_DEVICE 3 // Device
4 #define T_SYMLINK 4 // lab9: Soft symbolic link
```

```
xv6-labs-2021 > kernel >  fcntl.h
1 #define O_RDONLY 0x000
2 #define O_WRONLY 0x001
3 #define O_RDWR 0x002
4 #define O_CREATE 0x200
5 #define O_TRUNC 0x400
6 #define O_NOFOLLOW 0x004 // lab9
```

在 kernel/sysfile.c 中实现 sys_symlink() 函数，用于创建符号链接文件并将目标路径写入链接文件的 inode 中。

```
xv6-labs-2021 > kernel >  sysfile.c
488 // lab9: sys_symlink函数
489 uint64 sys_symlink(void) {
490     char target[MAXPATH], path[MAXPATH];
491     struct inode *ip;
492     int n;
493
494     if ((n = argstr(0, target, MAXPATH)) < 0
495         || argstr(1, path, MAXPATH) < 0) {
496         return -1;
497     }
498
499     begin_op();
500     // 创建symlink's inode
501     if((ip = create(path, T_SYMLINK, 0, 0)) == 0) {
502         end_op();
503         return -1;
504     }
505     // 向inode写入目标路径
506     if(writei(ip, 0, (uint64)target, 0, n) != n) {
507         iunlockput(ip);
508         end_op();
509         return -1;
510     }
511 }
```

```

xv6-labs-2021 > kernel > C sysfile.c
286 // lab9: recursively follow the symlinks
287 // Caller must hold ip->lock
288 // and when function returned, it holds ip->lock of returned ip
289 static struct inode* follow_symlink(struct inode* ip) {
290     uint inums[NSYMLINK];
291     int i, j;
292     char target[MAXPATH];
293
294     for(i = 0; i < NSYMLINK; ++i) {
295         inums[i] = ip->inum;
296         // read the target path from symlink file
297         if(readi(ip, 0, (uint64)target, 0, MAXPATH) <= 0) {
298             iunlockput(ip);
299             printf("open_symlink: open symlink failed\n");
300             return 0;
301         }
302         iunlockput(ip);
303
304         // get the inode of target path
305         if((ip = namei(target)) == 0) {
306             printf("open_symlink: path \"%s\" is not exist\n", target);
307             return 0;
308         }
309         for(j = 0; j <= i; ++j) {

```

修改 kernel/sysfile.c 中的 sys_open() 函数，添加对符号链接的处理，递归查找符号链接的目标文件并处理成环检测。

```

xv6-labs-2021 > kernel > C sysfile.c
327 sys_open(void){
363
364     // lab9: handle the symlink
365     if(ip->type == T_SYMLINK && (omode & O_NOFOLLOW) == 0) {
366         if((ip = follow_symlink(ip)) == 0) {
367             end_op();
368             return -1;
369         }
370     }

```

```

xv6-labs-2021 > M Makefile
174 UPROGS=\
186     $U/_stressts\
187     $U/_usertests\
188     $U/_grind\
189     $U/_wc\
190     $U/_zombie\
191     $U/_symlinktest\
192

```

2、测试：

在 xv6 中执行 symlinktest 测试：

```

xv6 kernel is booting

init: starting sh
$ symlinktest
Start: test symlinks
open_symlink: path "/testsymlink/a" is not exist
open_symlink: links form a cycle
test symlinks: ok
Start: test concurrent symlinks
test concurrent symlinks: ok
$

```

在 xv6 中执行 usertests 测试：

```
PROBLEMS OUTPUT DEBUG CONSOLE TESTS
test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
$
```

./grade-lab-fs symlinktest 单项测试:

```
canyon@Rikka:~/xv6-labs-2021$ ./grade-lab-fs symlinktest
make: 'kernel/kernel' is up to date.
== Test running symlinktest == (1.1s)
== Test   symlinktest: symlinks ==
    symlinktest: symlinks: OK
== Test   symlinktest: concurrent symlinks ==
    symlinktest: concurrent symlinks: OK
```

9.4. 实验心得

通过本次实验，我深入理解了 xv6 文件系统的结构和运作机制，尤其是在实现二级间接块索引的过程中，对 inode 的数据存储方式有了更深入的认识。此外，符号链接的实现让我对文件系统中的文件引用和递归操作有了更直观的理解。虽然实验中遇到了一些问题，例如 virtio_disk_intrstatus 的 panic，但通过调整代码，我最终成功解决了这些问题。这次实验不仅强化了我对文件系统概念的理解，还提升了我在实际操作系统开发中的调试和问题解决能力。

10. Lab10 Mmap

10.1. 实验目的

本实验的主要目的是在 xv6 操作系统中实现 mmap 和 munmap 系统调用，使用户进程能够将文件映射到虚拟内存中，并能够灵活地管理这些映射。具体地，实验要求实现 mmap 的内存映射功能，并支持懒分配和 MAP_SHARED/MAP_PRIVATE 标志的处理。同时，实现 munmap 系统调用，以取消映射并根据需求将修改写回文件。

10.2. 实验内容

1、mmap 系统调用的实现：

通过 `mmap` 系统调用，用户可以将文件的部分或全部内容映射到虚拟内存地址空间中。支持懒分配，即在发生页面错误时才实际分配物理内存。
支持 `MAP_SHARED` 和 `MAP_PRIVATE` 两种映射标志，分别表示共享映射和私有映射。

2、munmap 系统调用的实现：

通过 `munmap` 系统调用，用户可以取消映射，并在 `MAP_SHARED` 标志下将修改写回文件。

10.3. 实验步骤

1、修改 Makefile 和系统调用定义：

在 `Makefile` 中添加 `_mmaptest` 目标，以支持实验测试程序的编译。

```
xv6-labs-2021 > M Makefile
174     UPROGS=\
189         $U/_wc\
190         $U/_zombie\
191         $U/_mmaptest\
192
```

添加 `mmap` 和 `munmap` 系统调用的声明和定义，包括在 `kernel/syscall.h`、`kernel/syscall.c`、`user/usys.pl` 和 `user/user.h` 中进行相应修改。

```
xv6-labs-2021 > kernel > C syscall.h
22     #define SYS_close    21
23     #define SYS_mmap     22    // lab10
24     #define SYS_munmap   23    // lab10
```

```
xv6-labs-2021 > kernel > C syscall.c
105     extern uint64 sys_write(void);
106     extern uint64 sys_uptime(void);
107     extern uint64 sys_mmap(void);    // lab10
108     extern uint64 sys_munmap(void);  // lab10
109
```

```
xv6-labs-2021 > kernel > C syscall.c
131     [SYS_close]    sys_close,
132     [SYS_mmap]     sys_mmap,    // lab10
133     [SYS_munmap]   sys_munmap,  // lab10
134     ];
```

```
xv6-labs-2021 > user > usys.pl
37     entry("sleep");
38     entry("uptime");
39     entry("mmap");    # lab10
40     entry("munmap");  # lab10
```

```
xv6-labs-2021 > user > C user.h
24     int sleep(int);
25     int uptime(void);
26     void *mmap(void *addr, int length, int prot, int flags, int fd, int offset); // lab10
27     int munmap(void *addr, int length); // lab10
```

2、定义 Virtual Memory Area (VMA) 结构体:

在 kernel/proc.h 中定义 struct vm_area 结构体, 用于表示映射的虚拟内存区域。
在 struct proc 中添加 VMA 数组, 记录每个进程的虚拟内存映射区域。使用 NVMA 宏定义数组大小为 16。

```
xv6-labs-2021 > kernel > C proc.h
83  enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
84
85  // lab10: Virtual Memory Area
86  struct vm_area {
87      uint64 addr;    // mmap address
88      int len;       // mmap memory length
89      int prot;      // permission
90      int flags;     // the mmap flags
91      int offset;    // the file offset
92      struct file* f; // pointer to the mapped file
93  };
94
95  #define NVMA 16    // lab10: the number of VMA in a process
96
```

```
xv6-labs-2021 > kernel > C proc.h
98  struct proc {
119      char name[16]; // Process name (debugging)
120      struct vm_area vma[NVMA]; // lab10: VMA array
121  };
122
```

3、实现 mmap 系统调用:

在 kernel/sysfile.c 中实现 sys_mmap(), 处理参数提取、检查、VMA 分配和记录映射地址等操作。(注意: 记得在 memlayout.h 中添加如下定义)

```
xv6-labs-2021 > kernel > C memlayout.h
67  #define TRAPFRAME (TRAMPOLINE - PGSIZE)
68
69  // lab10: the minimum address the mmap can use
70  #define MMAPMINADDR (TRAPFRAME - 10 * PGSIZE)
```

```
xv6-labs-2021 > kernel > C sysfile.c
490
491
492  // lab10: 创建mmap函数
493  uint64 sys_mmap(void) {
494      uint64 addr;
495      int len, prot, flags, offset;
496      struct file *f;
497      struct vm_area *vma = 0;
498      struct proc *p = myproc();
499      int i;
500
501      if (argaddr(0, &addr) < 0 || argint(1, &len) < 0
502          || argint(2, &prot) < 0 || argint(3, &flags) < 0
503          || argfd(4, 0, &f) < 0 || argint(5, &offset) < 0) {
504          return -1;
505      }
506      if (flags != MAP_SHARED && flags != MAP_PRIVATE) {
507          return -1;
508      }
509      // the file must be written when flag is MAP_SHARED
510      if (flags == MAP_SHARED && f->writable == 0 && (prot & PROT_WRITE)) {
511          return -1;
512      }
513      // offset must be a multiple of the page size
```

支持懒分配，即在发生页面错误时，动态分配内存，并从文件中读取数据。
确保映射的地址由内核自行选择，并处理不同映射标志的情况。

4、实现页面错误处理：

在 kernel/trap.c 中的 usertrap() 函数中，添加对 mmap 映射内存的页面错误处理逻辑。

```
xv6-labs-2021 > kernel > C trap.c
41  usertrap(void){
56  if(r_scause() == 8){
69
70      syscall();
71  } else if (r_scause() == 12 || r_scause() == 13
72  || r_scause() == 15) { // lab10: mmap page fault - lab10
73      char *pa;
74      uint64 va = PGROUNDDOWN(r_stval());
75      struct vm_area *vma = 0;
76      int flags = PTE_U;
77      int i;
78      // find the VMA
79      for (i = 0; i < NVMA; ++i) {
80          // like the Linux mmap, it can modify the remaining bytes in
81          // the end of mapped page
82          if (p->vma[i].addr && va >= p->vma[i].addr
83              && va < p->vma[i].addr + p->vma[i].len) {
84              vma = &p->vma[i];
85              break;
86          }
87      }
```

根据页面错误的类型分配物理内存，并从文件中读取数据填充页面。
支持根据映射权限设置页面表项的标志位。

5、实现 munmap 系统调用：

在 kernel/sysfile.c 中实现 sys_munmap(), 处理参数提取、VMA 查找、取消映射和写回文件等操作。

```
xv6-labs-2021 > kernel > C vm.c
166  uvmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free){
173  for(a = va; a < va + npages*PGSIZE; a += PGSIZE){
174      if((pte = walk(pagetable, a, 0)) == 0)
175          panic("uvmunmap: walk");
176      if((*pte & PTE_V) == 0) {
177          continue; // lab10 delete
178      }
179      if(PTE_FLAGS(*pte) == PTE_V) {
180          continue; // lab10 delete
181      }
182      if(do_free){
183          uint64 pa = PTE2PA(*pte);
184          kfree((void*)pa);
185      }
186      *pte = 0;
187  }
```

```

xv6-labs-2021 > kernel > C sysfile.c
554 // lab10: 创建sys_munmap函数
555 uint64 sys_munmap(void) {
556     uint64 addr, va;
557     int len;
558     struct proc *p = myproc();
559     struct vm_area *vma = 0;
560     uint maxsz, n, n1;
561     int i;
562
563     if (argaddr(0, &addr) < 0 || argint(1, &len) < 0) {
564         return -1;
565     }
566     if (addr % PGSIZE || len < 0) {
567         return -1;
568     }
569
570     // find the VMA
571     for (i = 0; i < NVMA; ++i) {
572         if (p->vma[i].addr && addr >= p->vma[i].addr
573             && addr + len <= p->vma[i].addr + p->vma[i].len) {
574             vma = &p->vma[i];
575             break;
576         }
577     }

```

支持 MAP_SHARED 映射的写回操作，并确保在取消映射时正确释放资源。

6、修改 exit()与 fork()的逻辑，需要提前实现与处理脏页相关的两个函数。

```

xv6-labs-2021 > kernel > C riscv.h
341 #define PTE_V (1L << 0) // valid
342 #define PTE_R (1L << 1)
343 #define PTE_W (1L << 2)
344 #define PTE_X (1L << 3)
345 #define PTE_U (1L << 4) // 1 -> user can access
346 #define PTE_D (1L << 7) // dirty flag - lab10
347

```

```

xv6-labs-2021 > kernel > C vm.c
437 // lab10: get the dirty flag of the va's PTE
438 int uvmgetdirty(pagetable_t pagetable, uint64 va) {
439     pte_t *pte = walk(pagetable, va, 0);
440     if(pte == 0) {
441         return 0;
442     }
443     return (*pte & PTE_D);
444 }
445
446 // lab10: set the dirty flag and write flag of the va's PTE
447 int uvmsetdirtywrite(pagetable_t pagetable, uint64 va) {
448     pte_t *pte = walk(pagetable, va, 0);
449     if(pte == 0) {
450         return -1;
451     }
452     *pte |= PTE_D | PTE_W;
453     return 0;
454 }

```

```

xv6-labs-2021 > kernel > C proc.c
340 exit(int status){
    // lab10: unmap the mapped memory
    for (i = 0; i < NVMA; ++i) {
        if (p->vma[i].addr == 0) {
            continue;
        }
        vma = &p->vma[i];
        if ((vma->flags & MAP_SHARED)) {
            for (va = vma->addr; va < vma->addr + vma->len; va += PGSIZE) {
                if (uvmgetdirty(p->pagetable, va) == 0) {
                    continue;
                }
                n = min(PGSIZE, vma->addr + vma->len - va);
                for (r = 0; r < n; r += n1) {
                    n1 = min(maxsz, n - i);
                    begin_op();
                    ilock(vma->f->ip);
                    if (writei(vma->f->ip, 1, va + i, va - vma->addr + vma->offset + i, n1) != n1) {
                        iunlock(vma->f->ip);
                        end_op();
                        panic("exit: writei failed");
                    }
                    iunlock(vma->f->ip);
                }
            }
        }
    }
}

```

```

xv6-labs-2021 > kernel > C proc.c
273 fork(void){
    // lab10: copy all of VMA
    for (i = 0; i < NVMA; ++i) {
        if (p->vma[i].addr) {
            np->vma[i] = p->vma[i];
            filedup(np->vma[i].f);
        }
    }
    safestrncpy(np->name, p->name, sizeof(p->name));
    pid = np->pid;
    np->state = RUNNABLE;
    release(&np->lock);
    return pid;
}

```

7、测试:

在 xv6 中执行 mmaptest 测试:

```

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ mmaptest
mmap_test starting
test mmap f
test mmap f: OK
test mmap private
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test mmap two files
test mmap two files: OK
mmap_test: ALL OK
fork_test starting
fork test OK
mmaptest: all tests succeeded
$

```

xv6 中执行 usertests 测试:

```
sepc=0x00000000000002376 stva
OK
test opentest: OK
test writetest: OK
test writebig: OK
test createtest: OK
test openiput: OK
test exitiput: OK
test iput: OK
test mem: OK
test pipe1: OK
test killstatus: OK
test preempt: kill... wait... OK
test exitwait: OK
test rmdot: OK
test fourteen: OK
test bigfile: OK
test dirfile: OK
test iref: OK
test forktest: OK
test bigdir: OK
ALL TESTS PASSED
$
```

10.4. 实验心得

通过本次实验，我深入理解了 `mmap` 和 `munmap` 系统调用在操作系统中的实现原理。特别是在实现过程中，我学会了如何在内核中管理用户进程的虚拟内存，并处理懒分配、页面错误和写回文件等复杂操作。此外，实验还让我意识到在设计和实现系统调用时，需要充分考虑系统的性能和资源管理问题，如内存的懒分配和文件引用计数的管理。

实验还让我了解到 `VMA` 结构在管理进程的虚拟内存空间中的重要作用，并掌握了基本的内存管理技巧。虽然实现过程中遇到了一些困难，例如如何正确处理页面错误并确保映射区域的正确性，但通过查阅文档和调试代码，这些问题最终得以解决。

总体而言，这次实验不仅加强了我对操作系统内核开发的理解，还提升了我在实际编程中的问题解决能力。这将对我未来的学习和开发工作带来积极的影响。